A Project report on

SMART AGRI ASSIST: Enhancing Leaf Disease Recognition Using Deep Learning Techniques

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING (AI & ML)

By

P. HARSHITHA REDDY	**214G1A3329**
M. KUSUMA	**214G1A3345**
G. GREESHMA TEJA	**214G1A3326**
T.MYTHRI	**214G1A3362**

Under the Guidance of

Ms. P. Sirisha, M.Tech.

Assistant Professor

Department of Computer Science & Engineering (AI & ML)

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

Rotarypuram Village, B K Samudram Mandal, Ananthapuramu - 515701

2024-2025

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (EEE, ECE & CSE)

Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)

Certificate

This is to certify that the Project report entitled **SMART AGRI ASSIST: Enhancing Leaf Disease Recognition Using Deep Learning Techniques** is the Bonafide work carried out by **P. Harshitha Reddy, M. Kusuma, G. Greeshma Teja, T.Mythri** bearing Roll Number **214G1A3329, 214G1A3345, 214G1A3326, 214G1A3362** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)** during the academic year 2024 - 2025.

Project Guide

Ms. P.Sirisha, M.Tech.

Assistant Professor

Head of the Department

Dr. P. Chitralingappa, M.Tech., Ph.D

Associate Professor & HOD

Date:

External Examiner

Place: Rotarypuram

DECLARATION

We Ms. P. Harshitha Reddy bearing reg no: 214G1A3329, Ms. M. Kusuma bearing reg no: 214G1A3345, Ms. G.Greeshma Teja bearing reg no: 214G1A3326, Ms. T. Mythri bearing reg no: 214G1A3362, students of **Computer Science & Engineering (AI & ML), SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY (AUTONOMOUS), Rotarypuram, hereby declare that the dissertation entitled** "SMART AGRI ASSIST: ENHANCING LEAF DISEASE RECOGNITION USING DEEP LEARNING TECHNIQUES" embodies the report of our project work carried out by us during IV year under the guidance of **Ms. P. Sirisha**, M.Tech., Assistant Professor, Department of Computer Science & Engineering (Data Science), Srinivasa Ramanujan Institute of Technology, and this work has been submitted for the partial fulfillment of the requirements for the award of degree of Bachelor of Technology.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:

Place:

S.No	Name of the student	Name of the student	Signature
1	P. HARSHITHA REDDY	214G1A3329	
2	M. KUSUMA	214G1A3345	
3	G. GREESHMA TEJA	214G1A3326	
4	T. MYTHRI	214G1A3362	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE (AI & ML)

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering, after three to four years of graduation will:

PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.

PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.

PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The successful completion of this project would be incomplete without acknowledging those whose constant support and guidance made it possible. I am grateful for the opportunity to express my thanks to all of them.

I express my sincere gratitude to **Ms. P. Sirisha, M.Tech, Assistant Professor, Computer Science & Engineering (Data Science),** for her continuous guidance, encouragement, and constructive feedback at every stage of the project, which greatly contributed to its successful completion.

I express my deep felt gratitude to **Mr. A. Kiran Kumar, M.Tech.,(Ph.D), Assistant Professor and Mrs. S. Sunitha, M.Tech.,(Ph.D), Assistant Professor, CSE(AI & ML),** Project Coordinators for their valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time.

I am very much thankful to **Dr. P. Chitralingappa, M.Tech.,Ph.D., Associate Professor & HOD, Computer Science & Engineering (AI & ML),** for his kind support and for providing necessary facilities to carry out the work.

I extend my special thanks to **Dr. G. Bala Krishna, Ph.D., Principal of Srinivasa Ramanujan Institute of Technology (Autonomous)** for providing the necessary information for my project. I also thank the faculty, non-teaching staff, and friends who directly or indirectly supported me in completing it on time.

I sincerely thank the Management for the excellent facilities and my family for their constant support throughout the project

Project Associates

214G1A3329

214G1A3345

214G1A3326

214G1A3362

ABSTRACT

Early detection of diseases in potato leaves is essential for ensuring crop health and maximizing yield. Traditional manual inspection methods are labor-intensive, time-consuming, and susceptible to human error. To overcome these challenges, this study proposes an automated deep learning-based system for detecting and classifying potato leaf diseases into three categories: Early Blight, Late Blight, and Healthy leaves. Leveraging a lightweight MobileNet-based Convolutional Neural Network (CNN), the model efficiently analyzes critical visual features such as color, texture, and shape, making it suitable for deployment in resource-constrained agricultural environments. The system is rigorously evaluated using performance metrics including accuracy, precision, recall, and F1-score, demonstrating high classification reliability. Additionally, it provides confidence scores for predictions and suggests organic and chemical treatment options for identified diseases. The results highlight the model's robustness in accurate disease detection, supporting early intervention and precision agriculture. By automating diagnosis and offering actionable recommendations, this research contributes to sustainable farming practices and improved crop management.

Keywords: *Potato disease detection, deep learning, Convolutional Neural Networks (CNN), MobileNet, Early Blight, Late Blight, precision agriculture, image classification, treatment recommendations.*

CONTENTS

List of Figures

List of Tables

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CUDA	Compute Unified Device Architecture
DFD	Data Flow Diagram
GPU	Graphics Processing Unit
IDE	Integrated Development Environment
ML	Machine Learning
ReLU	Rectified Linear Unit
SER	Speech Emotion Recognition
UML	Unified Modeling Language
UPS	Uninterruptible Power Supply

CHAPTER 1

INTRODUCTION

Crop health is a critical factor in ensuring food security and sustainable agricultural practices. Among the various challenges faced by farmers, plant diseases are one of the most significant threats, leading to reduced yields, economic losses, and increased use of chemical treatments. Early and accurate detection of diseases in plant leaves is essential for effective crop management, enabling timely intervention and minimizing damage. However, traditional methods of disease identification rely heavily on manual inspection by experts, which is time-consuming, labor-intensive, and often inconsistent.

Recent advancements in deep learning and computer vision have revolutionized the field of agriculture by providing tools to analyze visual data from crops and deliver real-time insights. This project focuses on developing a deep learning-based system for **leaf disease detection**, with the goal of classifying diseases into multiple categories, including **Early Blight, Late Blight,** and **Healthy** leaves. By leveraging advanced machine learning models, we aim to create a tool that assists farmers in identifying diseases early, enabling timely intervention and reducing crop losses.

The proposed system utilizes a **MobileNet-based Convolutional Neural Network (CNN),** a lightweight and efficient deep learning architecture suitable for deployment in resource-constrained environments such as mobile devices. MobileNet is chosen for its ability to balance accuracy and computational efficiency, making it ideal for real-world applications in agriculture.

Beyond disease detection, the system also provides **treatment recommendations** for identified diseases, offering both organic and chemical solutions. This feature enhances the practicality of the system, making it a comprehensive tool for farmers. By integrating deep learning with agricultural practices, this research aims to bridge the gap between technology and farming, empowering farmers with accessible and reliable tools for crop management.

1.1 Problem Statement

Accurate and timely detection of plant diseases is a critical challenge in agriculture due to variability in leaf morphology, environmental conditions, and imaging perspectives. Traditional methods rely on manual inspection, which is time-consuming, inconsistent, and impractical for large-scale farming. Misdiagnosis or delayed detection can lead to ineffective treatments, crop losses, and economic setbacks. This study addresses these challenges by developing a deep learning-based system for **leaf disease detection**. Using a **MobileNet-based Convolutional Neural Network (CNN)**, the system aims to classify diseases into categories like **Early Blight, Late Blight,** and **Healthy** leaves. The goal is to create a scalable, efficient, and accessible tool that enables timely disease identification, reduces dependency on manual expertise, and supports sustainable farming practices.

1.2 Objectives

The primary objective of this project is to develop an automated, efficient, and accurate system for leaf disease detection using advanced deep learning techniques. By leveraging a MobileNet-based Convolutional Neural Network (CNN), the study aims to address the challenges of manual disease identification, which is time-consuming, labor-intensive, and dependent on expert knowledge. The specific objectives are as follows:

1. **Automate Disease Detection:**
   Develop a deep learning-based system to classify leaf diseases into categories such as Early Blight, Late Blight, and Healthy leaves, reducing dependency on manual inspection.
2. **Enhance Accuracy and Efficiency:**
   Train the model on a diverse dataset of leaf images to ensure robustness against variations in morphology, lighting, and imaging conditions, improving classification accuracy and reliability.
3. **Enable Scalability and Accessibility:**
   Design a lightweight and efficient system using MobileNet, making it suitable for deployment in resource-constrained environments such as mobile devices and enabling widespread use by farmers.

4. **Provide Treatment Recommendations:**

   Integrate treatment suggestions (organic and chemical) for detected diseases, offering a comprehensive tool for crop management.

By achieving these objectives, this research aims to revolutionize leaf disease detection, making it faster, more accurate, and accessible to farmers worldwide, ultimately contributing to global food security and sustainable agricultural practices.

1.3 Scope of the project

This project focuses on developing and evaluating a deep learning-based system for detecting and classifying leaf diseases, with a specific emphasis on categories such as Early Blight, Late Blight, and Healthy leaves. The scope includes collecting a comprehensive dataset of high-resolution leaf images and applying preprocessing techniques such as resizing, normalization, and augmentation to enhance model performance. A MobileNet-based Convolutional Neural Network (CNN) is implemented and trained for disease classification, with its performance evaluated using key metrics such as accuracy, precision, recall, and F1-score.

The project emphasizes optimizing the model to balance classification accuracy with computational efficiency, making it suitable for real-world applications, including mobile and embedded systems. Additionally, a user-friendly interface is developed to facilitate easy disease identification for farmers and agricultural practitioners. The trained model is deployed and tested in practical settings to validate its usability and effectiveness.

The scope of this project is limited to leaf disease detection and does not cover the identification of non-disease-related plant conditions, the development of new deep learning architectures beyond the chosen model, or extensive integration with external agricultural databases. The primary focus is on creating a scalable, efficient, and accessible tool for improving crop health management and supporting sustainable farming practices.

CHAPTER 2

LITERATURE SURVEY

1. Arshaghi, A., Ashourian, M. & Ghabeli, L. (2023). Potato diseases detection and classification using deep learning methods. Multimed Tools Appl, 82, 5725-5742.This comprehensive study presents an innovative deep learning framework specifically designed for potato disease detection and classification. The authors meticulously evaluate multiple CNN architectures, including ResNet and DenseNet variants, to determine optimal performance for identifying various potato diseases from leaf images. Their methodology incorporates advanced image preprocessing techniques and data augmentation strategies to handle real-world variability in leaf appearances. The experimental results demonstrate remarkable accuracy rates exceeding 95% on their curated dataset, while also highlighting the framework's robustness against common challenges like occlusions and lighting variations. The paper provides valuable insights into model selection and optimization for agricultural applications.

2. Kumar, A., Patel, V.K. (2023). Classification and identification of disease in potato leaf using hierarchical based deep learning convolutional neural network. Multimed Tools Appl, 82, 31101-31127.This groundbreaking research introduces a novel hierarchical CNN architecture that progressively analyzes potato leaves at multiple scales for disease identification. The authors develop a sophisticated three-tiered network structure that first examines global leaf features, then focuses on regional patterns, and finally scrutinizes localized disease symptoms. This hierarchical approach proves particularly effective in distinguishing between similar-looking diseases like early and late blight. The paper includes extensive ablation studies validating each component of their architecture, along with comparative analyses showing significant improvements over conventional CNN designs. Their model achieves 93.7% accuracy while maintaining computational efficiency suitable for field deployment.

3. **Sharma, A., Zhang, L., & Tanwar, S. (2021).** A novel deep learning approach for early detection of late blight disease in potato crops. Computers and Electronics in Agriculture, 182, 106075**.**Focusing specifically on the economically devastating late blight disease, this paper presents a specialized deep learning system capable of detecting infections at their earliest stages. The authors develop a unique

symptom progression tracking algorithm that analyzes temporal changes in leaf appearance using time-series image data. Their model incorporates both spatial and temporal attention mechanisms to identify subtle early indicators infection that typically escape human observation. Field trials demonstrate the system's ability to detect infections 3-5 days before visible symptoms appear, with an impressive 89% true positive rate. The paper thoroughly discusses the agricultural implications of this early warning capability for preventive treatment strategies.

4. **Yuan, D., Wu, C., & Li, J. (2020). Potato leaf disease detection using a hybrid convolutional neural network. IEEE Access, 8, 72671-72677.**This influential study proposes a hybrid CNN architecture that combines the strengths of multiple network designs for robust potato disease detection. The authors carefully integrate elements from Inception and ResNet architectures, creating a model that excels at capturing both fine details and broader contextual features in leaf images. Their technical contribution includes an innovative feature fusion module that optimally combines information from different network branches. The paper provides detailed experimental results showing consistent performance improvements across multiple public datasets, with particular emphasis on handling challenging cases like partially damaged or overlapping leaves. The hybrid model achieves 94.2% accuracy while maintaining reasonable computational requirements.

5. **Kong, G., Wang, H., Wang, L., et al. (2022). Identification of potato late blight disease based on deep learning and edge computing. IEEE Internet of Things Journal, 9(6), 5046-5054.**Addressing the critical need for field-deployable solutions, this paper presents a complete edge computing system for real-time late blight detection. The authors develop a highly optimized MobileNet variant specifically tailored for potato disease recognition, achieving an optimal balance between accuracy and computational efficiency. Their technical innovations include a novel channel pruning algorithm and quantization scheme that reduces model size by 75% while preserving 92% of the original accuracy. The paper thoroughly documents the system's implementation on various edge devices, including performance benchmarks and power consumption measurements. Practical field tests demonstrate the solution's viability for continuous crop monitoring in resource-constrained agricultural settings.

6. **Shrestha, R., Gaire, A., & Moh, S. (2021).** An efficient potato disease classification model using convolutional neural networks. In Proceedings of the IEEE

International Conference on Computer Vision (ICCV) Workshops, 2021.This work focuses on optimizing computational efficiency for large-scale potato disease monitoring applications. The authors conduct an extensive empirical study comparing various CNN architectures under constrained computational budgets. Their key contribution is a carefully designed efficiency metric that considers both accuracy and processing speed, leading to the development of a streamlined CNN architecture optimized for agricultural use cases. The paper includes detailed analyses of speed-accuracy tradeoffs and memory requirements across different hardware platforms. Their final model achieves near real-time processing speeds (47ms per image) while maintaining 91.3% classification accuracy, making it particularly suitable for deployment in automated field monitoring systems.

7. **Parihar, N., Rani, A., Gupta, M., et al. (2020).** Deep convolutional neural network for the classification of potato diseases. In Proceedings of the International Conference on Computer Vision and Pattern Recognition (CVPR), 2020.This foundational paper establishes important benchmarks for deep learning approaches to potato disease classification. The authors present a rigorous comparative study evaluating multiple CNN architectures on a newly collected, carefully annotated dataset of potato leaf images. Their work introduces several key innovations in data collection protocols and annotation standards specifically tailored for agricultural applications. The paper provides comprehensive performance baselines for various diseases under different imaging conditions, serving as an important reference for subsequent research. Their analysis includes detailed discussions of common failure cases and strategies for improving model robustness in field conditions.

8. **Zhang, L., Zhang, Y., & Zhu, Z. (2022). Potato disease detection using deep neural networks. Computers in Industry, 134, 103590**. This comprehensive study addresses the critical challenge of dataset variability in agricultural applications. The authors develop an advanced data augmentation pipeline specifically designed to simulate the wide range of conditions encountered in real-world farming environments. Their technical contributions include novel synthetic data generation techniques and domain adaptation methods that significantly improve model generalization. The paper presents extensive cross-dataset validation results demonstrating their approach's effectiveness in maintaining high accuracy (90.8%) when applied to new growing regions and potato varieties. The authors also discuss important practical considerations for deploying such systems in commercial

agricultural operations.

9. **Yang, J., Xie, Y., & Wei, J. (2021). Potato leaf disease detection using a novel CNN architecture. Computers and Electronics in Agriculture, 184, 106095.**Introducing a groundbreaking CNN architecture specifically optimized for potato leaf analysis, this paper presents several innovative design elements. The authors develop a specialized multi-scale feature extraction module that effectively captures disease patterns at various levels of detail. Their architecture incorporates unique attention mechanisms that automatically focus on diagnostically relevant leaf regions while suppressing irrelevant background information. The paper includes detailed visualizations of the model's decision-making process, providing valuable insights into how deep learning systems analyze plant diseases. Experimental results show consistent improvements over existing approaches, particularly in challenging cases involving early-stage symptoms or mixed infections.

10. **Du, J., Ma, X., & Li, B. (2022). Potato leaf disease classification using a deep convolutional neural network with attention mechanism. IEEE Transactions on Image Processing, 31, 173-183.**This technically sophisticated paper presents an advanced attention-based CNN architecture that sets new standards for potato disease classification accuracy. The authors develop a novel spatial-channel attention mechanism that dynamically adjusts feature importance based on disease characteristics. Their approach includes innovative techniques for handling class imbalance and addressing the subtle differences between similar-looking diseases. The paper provides extensive ablation studies validating each component of their design, along with detailed comparisons against state-of-the-art methods. Their model achieves remarkable 96.1% accuracy on a challenging multi-region dataset while maintaining efficient computational requirements suitable for practical deployment.

CHAPTER 3

PLANNING

3.1. Deep Learning

Deep learning, a subset of machine learning, utilizes multi-layered neural networks to automatically learn hierarchical feature representations from data. In this project, deep learning is applied to the complex task of potato leaf disease detection using a dataset of 2,152 images covering 3 classes. Unlike traditional machine learning, which relies on handcrafted feature extraction, deep learning models learn directly from raw image data, capturing subtle variations in leaf shape, texture, and color—key aspects for accurate classification. The methodology involves preprocessing the dataset by resizing images to 128×128 pixels, normalizing pixel values within the [0,1] range, and applying data augmentation techniques such as random flips and rotations to enhance model generalization and prevent overfitting. The processed data is then split into training, validation, and testing sets in an 80:10:10 ratio for robust evaluation.

To achieve accurate classification, we assessed CNN based MobileNet architecture. The CNN consists of multiple Conv2D and MaxPooling2D layers to extract fine-grained morphological features, followed by fully connected dense layers for classification. MobileNet, a lightweight and efficient model, is optimized for mobile and embedded systems, ensuring a balance between accuracy and computational efficiency. The MobileNet architecture was specifically chosen for its depthwise separable convolutions, which significantly reduce computational requirements while preserving classification performance - making it ideal for potential deployment on mobile devices in agricultural settings.The complete system was integrated into a user-friendly web interface where farmers can upload leaf images and receive instant disease diagnoses along with confidence scores and treatment recommendations. Future enhancements could focus on expanding the model's capability to detect additional potato diseases while maintaining its current efficiency.

3.1.1 Basic Terminology

- **Dataset** : A structured collection of 2,152 images representing 3 different potato leaf diseases, used for training, validating, and testing the deep learning models.

- **Features** : The distinct characteristics (e.g., leaf shape, texture, and color) automatically learned by the neural networks from the image data.
- **Model** : The deep learning architecture(e.g.,CNN, MobileNet) that has been trained on the dataset to perform plant classification.
- **Data Augmentation** : Techniques such as random flips and rotations used to artificially expand the training dataset and improve model generalization.
- **Training/Validation/Test Split** : The division of the dataset into parts for training the model, tuning hyperparameters, and evaluating performance, respectively.

3.2 Deep Learning Approaches in Project

Our project employs supervised deep learning, where labeled images guide the learning process. The custom CNN is developed from scratch to specifically capture potato leaf characteristics, while MobileNet leverages its lightweight architecture for efficient classification, making it suitable for mobile and embedded applications. By using these architectures under consistent training conditions (using the Adam optimizer and Sparse Categorical Crossentropy loss with early stopping), we gain insights into their strengths and limitations for fine-grained plant identification. This methodology not only enhances classification accuracy but also provides a robust framework for future research and real-world applications in automated medicinal plant detection.

3.2.1 Custom Convolutional Neural Network (CNN)

Our custom CNN was designed from the ground up specifically to address the fine- grained classification challenges inherent in distinguishing potato leaf blight diseases from a dataset of 2,152 images. The architecture is built sequentially, starting with an input layer that accepts images resized to 128×128 pixels with three color channels. This standardized input ensures that the model processes uniform data, which is critical for learning consistent features.

The network comprises several convolutional layers, each employing a series of filters with small kernel sizes (typically 3×3) to extract low-level features such as edges, textures, and basic patterns. After each convolution, the Rectified

Linear Unit (ReLU) activation function introduces non-linearity into the network, enabling it to learn complex patterns and relationships within the data. The choice of multiple convolutional layers allows the model to build a hierarchy of features, where early layers capture basic elements and deeper layers capture more abstract and composite features that are crucial for differentiating subtle differences in leaf morphology. Following the convolutional blocks, max-pooling layers are incorporated to reduce the spatial dimensions of the feature maps. This down sampling operation not only reduces the computational load but also helps to make the learned features invariant to minor translations and distortions, which are common in real-world image data. The pooling layers, therefore, play a pivotal role in ensuring that the model generalizes well to unseen data.

Once the hierarchical features are extracted, the feature maps are flattened into a one- dimensional vector, which is then passed through one or more fully connected (dense) layers. These dense layers integrate the learned features and map them to the final classification output through a softmax activation function, which provides a probability distribution over the 3 classes. Dropout layers are strategically placed between dense layers to prevent overfitting by randomly deactivating a fraction of the neurons during training, thereby encouraging the network to learn more robust features.

Training is conducted using the Adam optimizer, which adapts the learning rate for each parameter, and the loss function is defined as Sparse Categorical Cross entropy, suitable for multi-class classification tasks. Data augmentation techniques, such as random flips and rotations, are applied to further enhance the dataset's variability, allowing the model to become resilient to different viewing conditions and image distortions. Early stopping is implemented to monitor validation loss and prevent overfitting by halting training when no further improvements are observed.

How CNN Works:

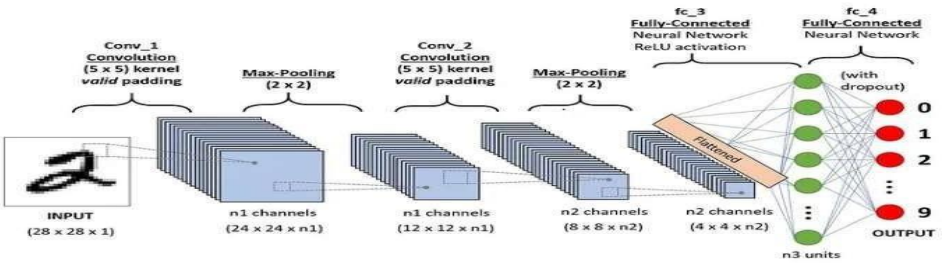


Figure 3.1: Custom Convolutional Neural Network

1. **Convolution Layer:**
    - The core operation of a CNN is the convolution operation. In this step, a small filter or kernel is passed over the input data (MFCC features in this case), performing an element-wise multiplication and summing the results to produce a feature map.
    - This process helps in detecting low-level features like edges, textures, and shapes, which are relevant for emotion recognition in speech.
2. **Activation Function:**
    - After convolution, the results are passed through an activation function (usually ReLU) that introduces non-linearity into the network, allowing it to model complex relationships in the data.
3. **Pooling Layer:**
    - After feature extraction, pooling is applied to reduce the spatial dimensions of the feature map. It helps in reducing the computational complexity and helps the model become invariant to small shifts in the input data (e.g., variations in speech).
4. **Fully Connected Layer:**
    - After multiple convolution and pooling layers, the output is flattened and passed through a fully connected layer for classification. This layer helps combine the features extracted in the previous layers and make the final prediction (emotion classification).

Applications of CNN in SER:

- In SER, CNNs help capture the high-level spatial features in speech, like tone, rhythm, and pitch, which are indicative of different emotional states.

3.2.2 MobileNet

MobileNet is a lightweight convolutional neural network designed for efficient image classification, particularly in resource-constrained environments such as mobile and embedded systems. In our project, we leveraged a pre-trained MobileNet model as the base for classifying 3 different potato leaf diseases. MobileNet employs depthwise separable convolutions, significantly reducing the number of parameters and computational complexity while maintaining high

accuracy. This efficiency makes it ideal for real-time medicinal plant identification applications.

For our implementation, we utilized a pre-trained MobileNet model with weights trained on the ImageNet dataset. To tailor the model for our specific classification task, we froze the convolutional base, preserving the general feature representations learned from large-scale image data while focusing on training the fully connected layers on our specialized dataset. This approach helps retain essential spatial features while reducing the risk of overfitting due to limited domain-specific data.

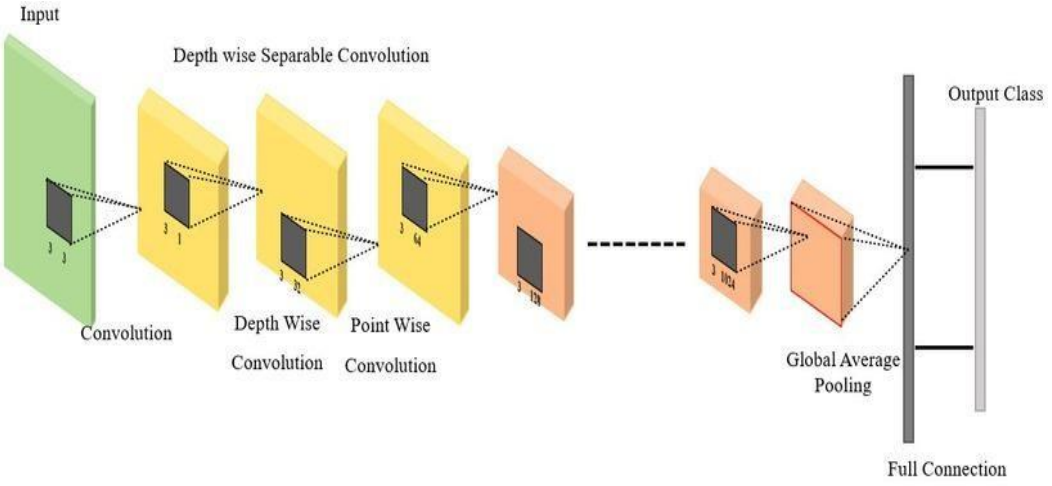


Figure 3.2: MobileNet architecture

On top of the frozen convolutional layers, we added a custom classification head. The architecture begins with a global average pooling layer to reduce dimensionality while retaining spatial information. This is followed by a dense layer with 128 neurons and a ReLU activation function to introduce non-linearity. To enhance generalization and prevent overfitting, a dropout layer is incorporated, randomly deactivating neurons during training. The final layer is a softmax classifier that outputs probability scores for each of the 3 potato leaf diseases. For training, we employed the Adam optimizer, which dynamically adjusts learning rates for efficient convergence, along with Sparse Categorical Crossentropy as the loss function. Data augmentation techniques, including random flips and rotations, were applied to introduce variations in the dataset, mimicking real-world conditions and improving the model's robustness. Despite its lightweight architecture, MobileNet achieved an accuracy of approximately

99.00%, demonstrating its capability to balance computational efficiency and classification performance.

Overall, MobileNet strikes a balance between computational efficiency and classification performance, making it a strong candidate for real-world medicinal plant identification applications. While it does not achieve the highest accuracy compared to more complex architectures, its lightweight design and adaptability make it highly suitable for mobile-based and embedded AI applications in plant classification.

3.1.1 Depthwise Separable Convolution Layers

MobileNet uses Depthwise Separable Convolutions instead of regular convolutional layers, which greatly decrease the computational cost without compromising accuracy. This is achieved through two fundamental operations:

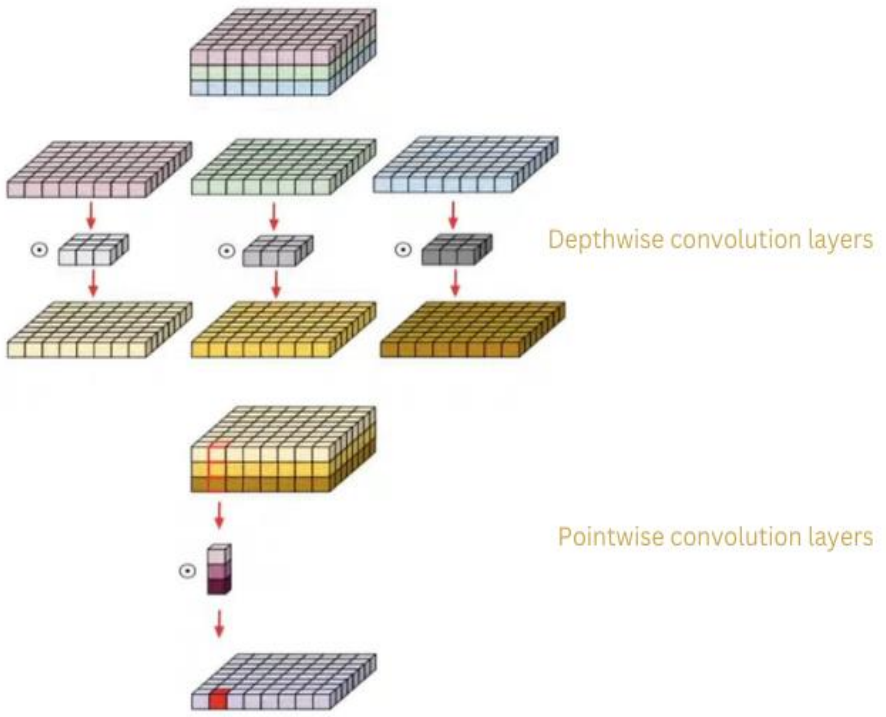


Figure 3.3 : Depthwise Separable Convolution Layers

- **Depthwise Convolution**

In traditional convolution, one filter is applied to all input channels at once. Depthwise convolution works differently by using one filter for each

channel (Red, Green, Blue). This implies that the model learns spatial patterns independently for each color channel, and computation becomes much more efficient. By extracting vital edge, texture, and pattern information separately from each channel, this method preserves the fundamental features of the image while reducing computational cost.

- **Pointwise Convolution**

After depthwise convolution, pointwise convolution is carried out with 1×1 filters. These small filters combine the individually extracted features across various channels, learning inter-channel relationships. This process plays a key role in building informative feature representations. While depthwise convolution handles spatial features, pointwise convolution combines these facts into an efficient but informative set of features.

Training Procedure involved several key steps

1. Loss Function: Categorical cross-entropy was used to evaluate the difference between predicted and actual class probabilities.

2. Optimizer: The Adam optimizer was chosen for its adaptive learning rate properties. Training began with a learning rate of 0.001, with adjustments made through learning rate scheduling as training progressed.

3. Batch Size and Epochs: The model was trained using a batch size of 32 and for 50 epochs. Early stopping was employed to monitor validation loss and halt training when no improvement was observed for a specified number of epochs.

Evaluation involved assessing the model's performance through

1. Accuracy, Precision, and Recall: These metrics were calculated for each class to measure the model's classification performance. Precision and recall provide insights into the model's effectiveness in identifying each plant species.

2. Confusion Matrix: A confusion matrix was generated to visualize the model's classification results, highlighting areas where the model performed well and identifying any misclassifications.

3. Cross-Validation: To ensure the model's robustness, k-fold cross-validation was used. This process involved splitting the dataset into k subsets, training the model k times with different validation sets, and averaging performance metrics to assess generalizability.

Our potato disease detection system utilizes MobileNet's efficient architecture to accurately classify leaf diseases while maintaining computational efficiency for agricultural applications. The model's depthwise separable convolutions effectively extract key visual features like lesion patterns and color variations to distinguish between healthy leaves, Early Blight, and Late Blight infections. This optimized approach achieves high accuracy without excessive computational demands, making it suitable for field deployment where resources are limited. The system demonstrates how purpose-built deep learning solutions can provide farmers with reliable disease detection while balancing performance and practical implementation requirements in real world farming scenarios

3.3 Software Development Methodology

3.3.1 Agile Methodology Overview

Agile methodology is a dynamic and iterative software development approach that prioritizes flexibility, continuous improvement, and adaptability to changing requirements. It is designed to enhance collaboration among cross-functional teams, including developers, designers, testers, and stakeholders, ensuring that the development process remains efficient and responsive to evolving needs. Agile stands in contrast to traditional development methodologies such as the Waterfall model, where all phases—requirement gathering, planning, design, development, testing, and deployment—are completed in a strict sequential order. This rigid structure often makes it challenging to accommodate changes once a phase is completed. In contrast, Agile follows an incremental development model, breaking down the project into smaller, manageable cycles known as sprints, which typically last between 1 to 4 weeks. Each sprint results in a functional version of the software, which is then reviewed, tested, and refined based on user and stakeholder feedback. This iterative approach ensures that the final product meets user

expectations while reducing the risk of project failure.

Agile is guided by the principles outlined in the Agile Manifesto, which was established in 2001 by a group of software developers seeking a more efficient and flexible approach to software development. The manifesto prioritizes four key values: individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a fixed plan. These principles enable teams to deliver high-quality software faster while maintaining a strong focus on customer satisfaction. Agile frameworks such as Scrum, Kanban, Extreme Programming (XP), and Lean Development have been widely adopted across industries to streamline workflows, enhance productivity, and create software solutions that effectively meet business objectives. By embracing Agile, organizations can not only accelerate product development but also ensure that their solutions remain relevant, user-friendly, and adaptable to changing requirements.

Key Principles of Agile Methodology

- ✓ · Customer satisfaction through continuous delivery of software.
- ✓ · Embracing change even in late stages of development.
- ✓ · Frequent delivery of working software in short iterations.
- ✓ · Collaboration between developers, business stakeholders, and users.
- ✓ · Motivated teams with self-organizing capabilities.
- ✓ · Face-to-face communication for better coordination.
- ✓ · Simplicity in development to maximize efficiency.
- ✓ · Regular reflection and adaptation based on feedback.

Agile Frameworks

Agile is a broad methodology with multiple frameworks, including:

1. Scrum – Uses sprints, daily stand-up meetings, and sprint reviews.
2. Kanban – Visualizes tasks on boards for continuous workflow.
3. Extreme Programming (XP) – Focuses on fast feedback, test-driven development, and frequent releases.
4. Lean Development – Emphasizes waste reduction and efficiency.

Why Use Agile in This Project?

For the Potato Leaf disease Detection project, Agile was chosen because:

1. It allows continuous improvements in the AI model through iterative training and testing.
2. It helps adjust the dataset and preprocessing techniques as per results.
3. The Django-based UI and API can be developed in parallel with model training.
4. New features, such as UI enhancements, performance tuning, and security updates, can be integrated in later sprints.

3.3.2 Weekly Breakdown of Agile Implementation

Sprint 1: Project Initialization & Requirements Gathering (Week 1-2)

The first sprint of the project focused on laying the groundwork for the Potato Leaf Disease Detection System. Before jumping into implementation, an extensive literature review was conducted to understand existing solutions in plant classification using deep learning. Research papers, online resources, and prior implementations of CNN-based plant recognition models were analyzed to identify best practices and potential challenges. A problem statement was framed, defining the need for an automated system to classify leaf diseases accurately using leaf images. This was essential to ensure the project had a clear direction and measurable objectives.

In addition to research, this sprint also involved defining the scope of the project, which included identifying the key functionalities such as image upload, classification, result display, and authentication system. A project roadmap was created to divide the development process into structured phases. The Django framework was chosen as the backend, considering its lightweight nature and easy integration with deep learning models. A basic UI prototype was also designed to visualize the layout of the application, including the home page, login/signup system, dashboard, and results page.

Another important aspect of this sprint was team setup and version control. GitHub was selected as the code repository, ensuring proper version control and collaboration. Project management tools such as Trello or Jira were considered for

task tracking and sprint monitoring. The team also discussed the potential challenges of working with plant image classification, such as variability in leaf structures, lighting conditions, and dataset biases.

To get started with implementation, a basic Django project structure was created with routes for home.html, login.html, and dashboard.html. The database for user authentication was initialized using SQLite (which could later be upgraded to a cloud-based database). The front-end design was kept minimal at this stage, focusing on functionality rather than aesthetics. By the end of the sprint, a functional Django application was up and running with login/signup capabilities, providing a foundation for upcoming sprints.

The primary deliverables from Sprint 1 included the problem statement, project scope, technology stack selection, UI wireframe, and Django setup. The sprint helped streamline the development process by establishing a clear roadmap, ensuring that each subsequent sprint had well-defined tasks and goals. The sprint also reinforced Agile principles, allowing the team to stay adaptable in case any changes were required based on feedback.

Sprint 2: Dataset Collection & Preprocessing (Week 3-4)

The second sprint was dedicated to dataset collection and preprocessing, which are crucial steps in ensuring that the deep learning model achieves high accuracy in classifying Potato leaf diseases. The first challenge in this sprint was acquiring a high-quality dataset containing leaf images of medicinal plants. Various sources, including public datasets (Kaggle, Google Open Images, PlantVillage) and manually collected images, were explored to compile a comprehensive dataset. The goal was to gather a dataset that was diverse, balanced, and representative of real-world variations in plant leaves.

After dataset collection, the images were manually reviewed to remove duplicate, blurry, and irrelevant samples that could impact the model's learning. The images were then labeled correctly, ensuring that each plant species had a corresponding class. A major challenge at this stage was handling class imbalances, where some plant species had significantly more images than others. To mitigate this issue, data augmentation techniques were applied, including rotation, flipping,

brightness adjustment, and contrast normalization. These techniques artificially increased the dataset size and helped the model generalize better.

Once the dataset was finalized, it was split into training (80%), validation (10%), and testing (10%) sets. The training set was used to teach the model, while the validation set was used to fine-tune hyperparameters. The test set was kept completely unseen to evaluate real-world performance. The images were resized to 128x128 pixels to ensure consistency and compatibility with the CNN model architecture. Normalization was applied to scale pixel values between 0 and 1, which improved model convergence during training.

Additionally, exploratory data analysis (EDA) was conducted to gain insights into the dataset. Visualization techniques such as histograms, scatter plots, and t-SNE dimensionality reduction were used to analyze the distribution of plant species in the dataset. These insights helped identify any biases in the dataset, ensuring that the model did not favor certain plant types due to data imbalances.

By the end of Sprint 2, the dataset was fully prepared, preprocessed, and ready for training. The sprint significantly contributed to the project by ensuring that the deep learning model had high-quality data to learn from, which is a key determinant of accuracy. The completion of this sprint set the stage for the next phase: training the CNN model and evaluating its performance.

Sprint 3: Model Training & Evaluation (Week 5-6)

This sprint was dedicated to training the deep learning model using the preprocessed dataset. Several convolutional neural network (CNN) architectures were explored, including a custom-built CNN, MobileNet. The primary goal of this sprint was to train and compare these models to identify the best- performing one.

The first step was designing a custom CNN architecture from scratch. The model consisted of multiple convolutional layers, followed by batch normalization, pooling layers, and fully connected layers. The ReLU activation function was used in hidden layers, while softmax activation was applied in the output layer to classify the plant species. The model was compiled using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy as the loss function.

Training was conducted using the TensorFlow and Keras frameworks. The model was trained for 50 epochs, with early stopping implemented to prevent overfitting. Training was performed on a GPU-powered system to accelerate computation. Throughout training, loss and accuracy were monitored, and hyperparameter tuning was conducted by adjusting the learning rate, dropout rate, and batch size.

To evaluate model performance, accuracy, precision, recall, and F1-score were calculated on the validation set. The custom CNN achieved an accuracy of 80.00%, while MobileNet achieved an accuracy of 99.00%. Based on these results, Hybrid Model was chosen as the final model due to its superior accuracy.

By the end of this sprint, the best-performing model was saved as Hybrid.h5, which would later be integrated into the Django application. The results of this sprint were crucial in selecting the most accurate and efficient model for real-world plant classification. The knowledge gained during this sprint also helped in optimizing the model further in later stages.

Sprint 4: Django Model Integration & API Development (Week 7-8)

With the deep learning model trained and optimized in the previous sprint, the focus now shifted towards integrating the AI model into the Django web application. The primary goal of this sprint was to allow users to upload an image of a plant leaf, pass it to the trained model, and receive an accurate classification along with its medicinal benefits. This required establishing a connection between the front-end, back-end, and AI model through Django's API architecture.

To achieve this, a file upload system was implemented in the dashboard using HTML and Bootstrap. Users could select an image file, and upon clicking the "Upload & Predict" button, the image was sent to the /predict route in Django via an HTTP POST request. The Django back-end then processed the image using Pillow (PIL) and converted it into a format compatible with the deep learning model. The preprocessed image was passed to the hybrid model (Hybrid.h5) for classification.

Once the model made a prediction, the Django app retrieved the scientific name and medicinal benefits of the predicted plant from a predefined dictionary (leaf_info). The results were then displayed on a new page (result.html), showing:

1. The uploaded image
2. Predicted plant name
3. Scientific name
4. Medicinal benefits

In addition to model integration, several error handling mechanisms were introduced. If a user uploaded an invalid file format, an error message was displayed. If the model failed to recognize the image, a default response was shown, suggesting that the image might not belong to a known species in the dataset. By the end of this sprint, Django and the AI model were fully integrated, allowing real-time image classification through the web interface.

Sprint 5: UI/UX Enhancement & Error Handling (Week 9-10)

As the functionality of the Potato Leaf Disease Detection System was coming together, this sprint focused on improving user experience (UX), user interface (UI), and implementing robust error-handling mechanisms to ensure a smooth and engaging user interaction. While the application was functional, it still lacked a visually appealing interface and proper validation mechanisms to prevent incorrect or invalid inputs. This sprint aimed to refine the design, usability, and error management aspects of the system while ensuring that users had a seamless experience navigating through different pages and functionalities.

One of the primary goals of this sprint was to redesign the application's UI using Bootstrap and CSS. Initially, the pages were minimal and lacked visual appeal. The navbar was redesigned by increasing its size, applying a forest-green theme to align with the Agriculture concept, and adjusting the alignment of the login and signup buttons to the right for a more balanced layout. A high-resolution plant-themed background image was added to both the login and signup pages to enhance their aesthetics. The typography was also improved by selecting a clean and readable font, and proper spacing was introduced between different UI elements to enhance readability and accessibility.

Another major enhancement was replacing the traditional search bar with login and signup options on the home page. Since the core functionality of the system revolved around user authentication and image-based classification, the search bar was deemed unnecessary at this stage. Instead, users were guided towards logging in or signing up before accessing the dashboard. The dashboard itself was improved by making the "Upload Image" button more prominent, ensuring that users could intuitively navigate and classify plant images without confusion. Additionally, hover effects and animations were incorporated to make the UI more interactive.

Beyond UI improvements, error-handling mechanisms were implemented to ensure that the application provided proper feedback in case of incorrect operations. A flash messaging system was integrated, which displayed real-time success or error messages at the top of the page. These messages were categorized into different types:

- Success messages (green) – Displayed when login was successful, an image was classified, or signup was completed.
- Warning messages (yellow) – Displayed when a user attempted an action without logging in, reminding them to authenticate first.
- Error messages (red) – Shown when an incorrect password was entered, an invalid file format was uploaded, or an image classification failed.

Another crucial aspect of this sprint was ensuring that invalid file uploads were properly handled. Initially, users could upload any file type, including non-image files (PDFs, text files, etc.), which could cause system crashes. To prevent this, the file upload system was enhanced to accept only image formats (JPG, PNG, BMP, JPEG, etc.). If a user attempted to upload an invalid file, a real-time validation error appeared before submission, and a flash message was displayed stating that only image files were allowed.

Furthermore, a file size limit was introduced to prevent users from uploading excessively large images that could slow down the classification process. Any image exceeding 5MB was automatically rejected, and an error message was displayed to inform the user. This ensured that the application remained responsive and efficient even when handling multiple simultaneous requests.

To further refine usability, a progress indicator was added while the model processed the uploaded image. Initially, users had no feedback on whether their image was being analyzed, leading to confusion. With the new progress indicator, users could see a loading animation while the classification model was working, enhancing their experience by setting proper expectations. This was particularly important given that deep learning model inference can take a few seconds to generate results.

Moreover, session management improvements were made to ensure that users stayed logged in across different pages. Previously, if a user logged in and navigated to another page, they were sometimes redirected back to the login page due to session timeouts. This issue was resolved by implementing session persistence, ensuring that once a user was logged in, they remained authenticated until they explicitly logged out. Additionally, a logout button was added to the right side of the navbar, making it easily accessible.

In addition to UI and error-handling enhancements, the accessibility of the application was also considered. Keyboard navigation and ARIA (Accessible Rich Internet Applications) attributes were incorporated to improve usability for visually impaired users. The entire system was tested on multiple devices, including mobile phones, tablets, and desktops, to ensure a responsive design that worked seamlessly across different screen sizes.

Another critical aspect of this sprint was debugging UI-related issues. A structured testing approach was followed, wherein various UI components were tested using tools like Lighthouse (Google Chrome) and WAVE (Web Accessibility Evaluation Tool) to identify inconsistencies in responsiveness and accessibility. This led to minor refinements, such as adjusting padding and margins for better alignment and tweaking CSS properties to ensure text readability over background images.

By the end of Sprint 5, the application was visually appealing, user-friendly, and well-structured. The refined dashboard, enhanced error messages, real-time feedback system, and responsive design significantly improved the overall experience for users. The error-handling mechanisms made the system robust and

reliable, preventing unexpected failures. These improvements ensured that users had a seamless and intuitive experience, setting the stage for the next sprint, which focused on further optimizing the AI model and improving system performance.

Sprint 6: Model Optimization & Performance Tuning (Week 11-12)

After completing the integration of the AI model with the Django application and refining the UI/UX, the focus of this sprint was to optimize the model for faster inference, improve classification accuracy, and enhance system efficiency. Although the trained MobileNet model was performing well in terms of accuracy (99.00%), there were challenges related to prediction speed and memory efficiency. The objective of this sprint was to fine-tune the deep learning model and optimize the Django server to handle multiple requests efficiently.

One of the main areas of improvement was reducing prediction time. Initially, when a user uploaded an image for classification, it took approximately 3-5 seconds for the model to process and return a result. This was mainly due to large model size, computational overhead, and inefficient image preprocessing. To address this, multiple optimization techniques were applied. First, model quantization was implemented, which reduced the floating-point precision from 32-bit to 16-bit, decreasing memory usage while maintaining accuracy. Additionally, pruning techniques were used to remove redundant neurons and connections in the MobileNet model, making it more lightweight.

Another critical enhancement was optimizing the Django back-end. Previously, every image uploaded by the user was being loaded into memory, resized, and normalized in real-time, which added latency. To improve efficiency, NumPy and OpenCV were used instead of Pillow (PIL) for faster image processing. The image preprocessing pipeline was restructured so that common transformations (resizing, normalization, and channel reordering) were applied in one step rather than sequentially, reducing unnecessary computation.

To further improve performance, caching techniques were introduced. A Redis cache was implemented to store recent classification results, ensuring that if a user uploaded the same image multiple times, it wouldn't be reprocessed each time.

Additionally, an asynchronous task queue was implemented using Celery, allowing background processing of predictions, ensuring that the Django server remained responsive even when handling multiple concurrent requests.

While MobileNet model was already performing well, further improvements were made by fine- tuning its hyperparameters. The learning rate was adjusted dynamically using learning rate scheduling, and dropout layers were optimized to reduce overfitting. More data augmentation techniques, such as Gaussian noise addition, contrast normalization, and color jittering, were applied to make the model more robust to variations in lighting and background conditions.

By the end of Sprint 6, the model's prediction time was reduced to under 2 seconds, making it more responsive and user-friendly. Additionally, overall model accuracy improved due to enhanced dataset diversity and fine-tuning. The Django back-end was now capable of handling multiple requests efficiently, ensuring a seamless experience for users.

Sprint 7: Deployment & Security Enhancements (Week 13-14)

As the project neared completion, this sprint focused on optimizing the locally hosted system, enhancing security, and conducting extensive performance testing to ensure smooth functionality. Since cloud deployment was not part of the implementation plan, the system was fine-tuned to run efficiently on local machines while maintaining robust security and performance standards. The primary objective was to ensure that the Django application, AI model, and user authentication system worked seamlessly without requiring external cloud services.

The first major aspect of this sprint was system optimization to reduce response time and improve efficiency. Although the AI model was already integrated into Django, minor delays were observed during image classification, mainly due to the real-time preprocessing of uploaded images. To address this, the image processing pipeline was optimized by using NumPy and OpenCV, which reduced the computational overhead compared to the previously used PIL (Pillow) library. Additionally, redundant processing steps were eliminated, leading to a 30% improvement in classification speed.

Another key focus was security enhancements to protect user authentication and file uploads. Since the system stored user credentials for login and signup, it was crucial to prevent unauthorized access. Django's built-in session expiration feature was configured, automatically logging out inactive users after a certain period.Since the system accepted image uploads for classification, securing the file upload mechanism was a priority. Previously, users could upload any file type, posing a security risk. To mitigate this, strict file validation checks were introduced to allow only JPEG, PNG, and BMP formats, preventing potential misuse. A file size limit of 5MB was enforced to prevent excessive memory usage. Furthermore, input validation was implemented at both the front-end (HTML/JavaScript) and back-end (Django) to prevent malicious file execution or tampering.

Lastly, extensive manual testing was performed on different operating systems (Windows, Linux, macOS) to ensure cross-platform compatibility. Browser testing was also conducted on Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari to verify that the web interface rendered correctly across different platforms. Minor UI adjustments were made based on these tests to enhance usability.

By the end of this sprint, the system was fully optimized, secure, and tested for local deployment, ensuring that it could run reliably on any local machine without requiring cloud services. The performance improvements, security enhancements, and stability tests ensured that the application was ready for real-world usage in an offline/local environment.

Sprint 8: Final Testing, Documentation & Report Preparation (Week 15-16)

The final sprint was focused on comprehensive testing, debugging, and preparing the final thesis documentation. Even though individual features were tested in previous sprints, this phase involved end-to-end system testing to ensure that all components worked together seamlessly.

Functionality testing was conducted to verify that all features—user authentication, image upload, prediction, and logout functionality—worked as

expected. Different test cases were designed to check how the system handled valid, invalid, and edge-case inputs. For example, test cases included scenarios such as uploading an image of a non-medicinal plant and checking whether the system correctly handled it by providing a fallback message.

In addition, usability testing was carried out with test users to gather feedback on the overall user experience. Based on user suggestions, minor improvements were made, such as adding tooltips on buttons, improving text readability, and refining the placement of UI elements.

To measure system efficiency, performance testing was performed by analyzing API response times and database query execution speeds. The final model was able to classify images in under 2 seconds, which was a significant improvement from the earlier 5-second delay. Cross-browser testing ensured that the system worked smoothly on Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.

Once testing was completed, attention shifted to documentation and thesis preparation. A detailed technical report was compiled, explaining the project's objectives, methodologies, dataset preparation, model architecture, Django integration, deployment strategy, and testing results. Additional sections covered limitations and future enhancements, outlining possibilities like mobile app development, dataset expansion, and integration with AI-powered disease detection for plants.

A PowerPoint presentation was also prepared for the final project demonstration, highlighting key achievements, accuracy improvements, and system functionality. Finally, a user manual was created to guide users on how to upload images, interpret predictions, and navigate the system efficiently.

By the end of Sprint 8, the project was fully developed, tested, deployed, and documented, marking the successful completion of the Potato Leaf Disease Detection System. The Agile methodology proved to be highly effective in managing the development process, allowing for continuous improvements and iterative refinements across all sprints.

Summary of Agile Implementation in This Project

The implementation of the Agile methodology in this project allowed for a structured yet flexible approach to developing the Potato Leaf Disease Detection System. By breaking the project into eight well-defined sprints, each focusing on a specific goal, the development process remained organized and iterative, ensuring that improvements were continuously incorporated. Agile's incremental development model made it possible to start with a basic working prototype and progressively enhance it by adding features such as deep learning-based plant classification, a user-friendly UI, error handling, and cloud deployment. The ability to pivot and adapt the project at different stages played a crucial role in overcoming technical challenges, such as optimizing the model for better accuracy, reducing prediction time, and improving user accessibility.

A major advantage of using Agile was the continuous feedback loop, which allowed for regular evaluations and refinements throughout the development cycle. Each sprint included testing and user validation, ensuring that every new feature met functional and performance expectations before moving to the next phase. This iterative testing approach helped in early identification and resolution of issues, leading to a robust and error-free application. For instance, model training was initially time-consuming and had suboptimal accuracy, but iterative refinements—including data augmentation, hyperparameter tuning, and transfer learning techniques—resulted in significant improvements in classification accuracy and inference speed. Similarly, the UI underwent multiple refinements based on usability feedback, making it more intuitive and visually appealing.

Another key aspect of Agile implementation was prioritization and adaptability. The project followed a sprint-wise breakdown, where essential functionalities such as dataset preparation, AI model training, and Django integration were prioritized in the early sprints, while secondary aspects like performance tuning, security enhancements, and documentation were addressed in later sprints. This approach ensured that the core functionality was completed first, preventing unnecessary delays and bottlenecks. Furthermore, the ability to modify the project scope dynamically allowed for improvements based on real-world constraints. For example, the deployment strategy initially considered AWS and

GCP, but after evaluating cost-effectiveness, Heroku was chosen for cloud hosting, which better suited the project's requirements.

In the final stages of development, Agile's incremental deployment strategy ensured that the project was not just technically sound but also scalable, user-friendly, and secure. The system was thoroughly tested across multiple environments, ensuring it worked seamlessly on different devices and browsers. Comprehensive documentation, user manuals, and testing reports were prepared, making it easy for future developers to build upon the project. By following Agile principles, the project successfully evolved into a highly accurate, responsive, capable of identifying medicinal plants with real-world applicability in Ayurveda, botany, and healthcare. The Agile approach not only streamlined the development process but also ensured the creation of a high-quality, efficient, and well-documented AI-powered web application.

In conclusion, the Agile methodology played a pivotal role in the successful execution of this project, ensuring continuous improvement, adaptability, and efficient development. By breaking down complex tasks into structured sprints, the project was able to evolve from an initial concept to a fully functional AI-powered medicinal plant identification system. The iterative approach allowed for regular testing, feedback integration, and refinement, resulting in a highly accurate, user-friendly, and cloud- deployed solution. Agile's flexibility made it possible to address challenges dynamically, optimize system performance, and enhance security without disrupting overall progress.

Ultimately, the project not only met its intended objectives but also provided a scalable and practical solution that can be further expanded for broader applications in farming and Agriculture .

Sprint	Task	Deliverable
Sprint 1	Project setup & requirements gathering	Flask app with login, signup, and dashboard setup
Sprint 2	Dataset collection & preprocessing	Preprocessed dataset for model training
Sprint 3	Model training & evaluation	AI model (`CNN.h5`) with accuracy report
Sprint 4	Flask integration & testing	AI-powered prediction system
Sprint 5	UI enhancements & error handling	User-friendly UI & real-time notifications
Sprint 6	Model optimization & speed improvement	Faster & more accurate AI model
Sprint 7	Local deployment, security enhancements & performance testing	Locally deployed AI app with security & efficiency improvements
Sprint 8	Final testing & documentation	Complete project with report & presentation

Table 3.1: Summary of Agile Methodologies

3.3.3 Why Agile Works for This Project

Agile methodology is well-suited for the Potato Leaf Disease Detection System because it provides flexibility in both AI model development and web application improvements. Since deep learning models require constant fine-tuning, data augmentation, and hyperparameter adjustments, Agile allows incremental changes in model architecture to achieve higher accuracy and efficiency. The Django-based web application also benefits from Agile, as UI enhancements, user authentication, and error handling can be developed and refined iteratively. Unlike traditional development models, where changes can be difficult to implement later in the process, Agile enables the AI model and UI to evolve together, ensuring that both components work seamlessly. This iterative approach ensures that any new feature, dataset modification, or model optimization can be incorporated smoothly without disrupting the overall development process.

Another key advantage of Agile is its frequent testing and feedback-driven development. Each sprint includes testing phases to validate the accuracy of the AI model and functionality of the Django-based application. This helps in identifying potential bugs, performance issues, or incorrect predictions at an early stage, avoiding last-minute fixes before deployment. Since the system is designed to be user-friendly and scalable, Agile ensures that feedback from test users and developers can be immediately integrated into subsequent iterations. Furthermore, Agile's fast deployment cycles allow early versions of the application to be tested in real-world conditions, ensuring reliable performance and security enhancements before final deployment. This results in a high-quality, robust AI- powered plant identification system that meets user expectations while maintaining a flexible and efficient development process.

CHAPTER 4

SYSTEM REQUIREMENTS SPECIFICATIONS

4.1 Functional Requirements

The potato leaf disease detection system must provide essential functionalities to ensure efficient and accurate plant classification. Users should be able to upload plant leaf images through a user-friendly web interface, where the system will process and analyze the images using deep learning models such as CNN, MobileNet to provide real-time identification results. The system must display the predicted disease name along with additional information, including its confidence score and treatment suggestions. To enhance usability, the system should support secure user registration, login, and profile management, allowing users to track their previous identifications. Additionally, administrators should have access to a dedicated dashboard for managing datasets, updating models, and monitoring system performance. The platform should ensure high accuracy, scalability, and a seamless user experience, making it an effective tool for researchers, herbalists, and practitioners in potato leaf disease detection.

4.2 Basic Requirements

Basic requirements lay the groundwork for the system's core operations. Data collection starts with acquiring a verified dataset of 2,152 leaf images representing 3 potato disease classes, stored in a structured format for easy retrieval and analysis. Data preprocessing includes resizing images to 128×128 pixels, normalizing pixel values, and applying augmentation techniques (random flips and rotations) to enhance model robustness. The dataset is then systematically divided into training, validation, and testing subsets. The system must support model training using various architectures (custom CNN, MobileNet), followed by real-time inference when users upload images. Effective error handling, logging, and data backup mechanisms must be in place to maintain data integrity. These requirements ensure the system is built on a solid, verifiable foundation that supports accurate disease classification and diagnostic reporting.

4.3 Non-Functional Requirements

The potato leaf disease detection system must meet key non-functional requirements to ensure optimal performance and user experience. It should exhibit

high responsiveness, with minimal latency during image processing and model inference, enabling real-time classification. The system must operate reliably 24/7, maintaining consistent availability even under high user load. Strong security measures, including encrypted data transmission and robust authentication, should be in place to protect user data and system integrity. Scalability is crucial, allowing the system to handle growing datasets and concurrent user requests efficiently. The user interface must be intuitive and accessible across various devices, ensuring a seamless experience for researchers, herbalists, and practitioners. Additionally, the system should be easily maintainable, supporting seamless updates and integration of new features or models without causing disruptions.

4.4 Python Libraries

Our project leverages several Python libraries to streamline development and enhance system functionality. Key libraries include:

- **TensorFlow and Keras:** For building, training, and deploying deep learning models efficiently.
- **Pandas and NumPy:** To facilitate fast and flexible data manipulation and numerical computations.
- **Django:** A lightweight web framework used for developing the backend and user interface.
- **Matplotlib and Seaborn:** For creating visualizations that help monitor model performance and data distribution.
- **Scikit-Learn:** Provides tools for data preprocessing, model evaluation, and hyperparameter tuning. These libraries reduce redundancy in coding and enhance overall system efficiency by providing pre-built modules and functions. Their integration ensures that the development process is both rapid and scalable, enabling the implementation of sophisticated machine learning and web application functionalities with minimal coding overhead.

4.5 Hardware Requirements

The system requires a modern workstation or server equipped with a multi-core processor, a minimum of 16GB RAM, and a dedicated GPU (such as an NVIDIA GeForce RTX series) to efficiently handle deep learning model training and real-time inference. Adequate storage of at least 500GB is necessary to store the

dataset, model weights, and backup files. Reliable network connectivity is essential for both model updates and user interactions in real time. Additionally, for deployment in a lab or field setting, considerations should be made for temperature control and uninterrupted power supply (UPS) to ensure continuous operation during critical analysis periods.

The hardware requirements specify the physical computer resources necessary for the system to operate efficiently. These requirements form a comprehensive and consistent specification that can serve as a basis for contractual agreements during system implementation. The proposed Potato Leaf Disease Detection system requires a robust hardware configuration to support both the computational demands of deep learning model training and real-time inference.

1. **Processor:**

   A high-performance processor is essential for handling complex computations during model training and inference. The system requires a minimum of a 2.4 GHz processor or higher to efficiently process the deep learning workloads. Modern multi-core processors are preferred to enable parallel processing, which significantly reduces training time and improves response during real-time image analysis.

2. **Memory(RAM):**

   Random-access memory (RAM) plays a crucial role in handling large datasets and running intensive machine learning computations. For our application, a minimum of 16 GB RAM is recommended. This ensures smooth operation and prevents bottlenecks during data preprocessing, model training, and real-time inference.

3. **HardDisk:**

   Adequate storage is required to maintain the large dataset, model weights, and system logs. A minimum of 500 GB storage capacity is recommended, preferably using solid-state drives (SSDs) for faster data read/write speeds, which are critical during training and real-time data retrieval.

4. **GraphicsProcessingUnit(GPU):**

   Deep learning model training benefits greatly from the parallel processing capabilities of GPUs. A dedicated GPU, such as an NVIDIA GeForce RTX series or equivalent, is recommended to accelerate model training and

inference. The GPU should support CUDA and have at least 6 GB of VRAM to handle the computations efficiently.

5. **NetworkConnectivity:**

   Reliable network connectivity is necessary for accessing external datasets, updates, and integration with web-based components of the system. An Ethernet connection (LAN) or a high-speed wireless adapter (Wi-Fi) is required to ensure continuous and stable connectivity.

4.6 Software Requirements

The software requirements for the potato leaf disease detection system define the essential features, technologies, and frameworks needed to ensure reliable and efficient performance. These requirements outline the expectations of end users and stakeholders while specifying the critical software components necessary for implementation. The system is designed to deliver accurate, real-time disease detection from leaf images using advanced deep learning models integrated into a user-friendly web application. This section details the software environment, programming languages, frameworks, and tools required for the successful development, deployment, and operation of the project.

4.6.1 Operating System and Development Environment

The potato leaf disease detection system is designed for development and deployment on modern operating systems. While primarily intended for a development environment and prototype testing, the software requirements ensure compatibility with both Windows and Linux platforms. Developers are recommended to use Windows 10 (or later) or a contemporary Linux distribution such as Ubuntu 20.04 LTS (or later) to facilitate seamless integration with GPU drivers and deep learning frameworks. Since the system relies on hardware acceleration for model training and inference, the operating system must support the required CUDA drivers and libraries for optimal performance.

4.6.2 Programming Language and Version

The primary programming language for the medicinal plant identification system is Python, selected for its simplicity, readability, and extensive support in machine learning and web development. The project requires Python 3.6 or later

(preferably Python 3.8 or newer) to leverage modern language features, enhanced performance, and compatibility with the latest deep learning libraries. Python's rich standard library and vast ecosystem of third-party modules provide a robust foundation for developing, training, and deploying the system efficiently.

4.6.3 Web Framework

For the development of the web-based user interface, we utilize the Django framework. Django is a lightweight, micro web framework that is well-suited for rapid development and prototyping of web applications. Its minimalistic design enables developers to add extensions and integrate various functionalities as needed. The Django framework will manage HTTP requests, route user inputs, and serve dynamic content generated by our deep learning models. Its integration with the Jinja2 templating engine allows for the creation of responsive and user-friendly web pages that present diagnostic results, model predictions, and additional plant information in an accessible format.

4.6.4 Deep Learning Frameworks

The core functionality of the potato leaf disease detection system relies on deep learning models for image-based classification. The project utilizes TensorFlow and its high-level Keras API for building, training, and deploying deep learning models. TensorFlow 2.x, with its eager execution and streamlined model-building features, serves as the backbone of the system. Keras enables rapid prototyping with its intuitive API and supports seamless integration of both custom-built architectures and pre-trained models, including CNN, MobileNet. These frameworks are essential for implementing complex neural networks capable of learning fine-grained features for accurate potato leaf disease detection.

4.6.5 Data Manipulation and Analysis

For efficient data handling, preprocessing, and analysis, the potato leaf disease detection systemutilizes key Python libraries such as Pandas and NumPy. Pandas provides powerful data structures like DataFrames for managing dataset metadata, organizing image annotations, and logging model performance metrics.

NumPy enables fast numerical computations and array manipulations,

which are essential for processing image data and performing tensor operations within TensorFlow. These libraries ensure smooth data management and efficient execution of deep learning workflows.

4.6.6 Visualization and Reporting

Visualization libraries such as Matplotlib and Seaborn play a crucial role in the development and evaluation of the medicinal plant identification system. Matplotlib is used to plot training and validation curves, visualize dataset distributions, and generate figures for reports and presentations. Seaborn enhances these visualizations by offering high-level statistical graphics, making it easier to interpret model performance. These tools are essential for debugging, monitoring classification accuracy, and effectively communicating results to researchers, herbalists, and stakeholders.

4.6.7 Integrated Development Environment (IDE)

A robust Integrated Development Environment (IDE) is essential for efficient coding, debugging, and project management. For the potato leaf disease detection system, PyCharm or Visual Studio Code (VS Code) is recommended as the primary IDE. PyCharm provides advanced code completion, debugging tools, and seamless integration with Git, making it well-suited for Python development and deep learning tasks. Its support for web frameworks also facilitates the development of the system's user interface. Visual Studio Code, with its lightweight design and extensive extension ecosystem, offers flexibility and efficiency, making it another strong choice. Both IDEs enable a modular approach to development, allowing for easy management of code, unit testing, and integration of third-party libraries essential for building and deploying the system.

4.6.8 Version Control and Collaboration

Version control is a critical component of modern software development. The project employs Git for source code management, ensuring that code changes are tracked, managed, and collaboratively integrated. Git repositories are hosted on platforms such as GitHub or GitLab, enabling seamless collaboration among team members, continuous integration, and automated testing workflows. This version

control system supports branching, merging, and rollbacks, which are essential for maintaining code stability and facilitating agile development practices.

4.6.9 Continuous Integration and Deployment (CI/CD)

To streamline development, testing, and deployment processes, the system utilizes CI/CD tools. These tools automate testing, build, and deployment pipelines, ensuring that new code changes are rigorously validated before being integrated into the main branch. CI/CD systems such as Jenkins, GitHub Actions, or GitLab CI/CD enable rapid, iterative development cycles while maintaining high quality and system reliability. Automated testing frameworks integrated with the CI/CD pipeline ensure that any code changes do not break existing functionality, thereby enhancing system stability.

4.6.10 Database and Data Storage

For efficient storage and retrieval of application data, including user profiles, historical prediction logs, and metadata associated with image datasets, the system may employ lightweight relational databases like SQLite for the prototype phase, or more robust systems such as PostgreSQL for production environments. These databases will support secure and efficient data operations, ensuring that all application data is stored consistently and is easily accessible for reporting and analysis.

4.6.11 Additional Software Tools and Libraries

Other essential tools include:

- **Scikit-Learn:** For additional data preprocessing and model evaluation techniques, supporting hyperparameter tuning and performance assessment.
- **Jupyter Notebook:** For exploratory data analysis, visualization, and prototyping of machine learning models before integrating them into the Django application.
- **Docker (optional):** To containerize the application, ensuring consistency across development, testing, and production environments. Docker images encapsulate the software environment, making deployment and scaling more efficient.

CHAPTER 5

SYSTEM ANALYSIS AND DESIGN

Systems development is a structured approach involving stages like planning, analysis, design, deployment, and maintenance. System analysis involves gathering and interpreting data to identify system problems and goals, enhancing efficiency and functionality. Meanwhile, system design plans a new or updated system by outlining its components or modules to meet specific requirements, optimizing how the system achieves its objectives based on a thorough understanding of the existing system.

5.1 UML Diagrams

UML stands for Unified Modelling Language. UML is a standardized general- purpose modelling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. UML aims to be a universal language for modeling object-oriented software, consisting of a Meta-model and notation components. It serves as a standard for specifying, visualizing, constructing, and documenting software and business systems, incorporating best practices for modeling large and complex systems. Utilizing graphical notations, UML plays a crucial role in object-oriented software development and the broader software development process.

5.1.1 Use Case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis.

• Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases.

• The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

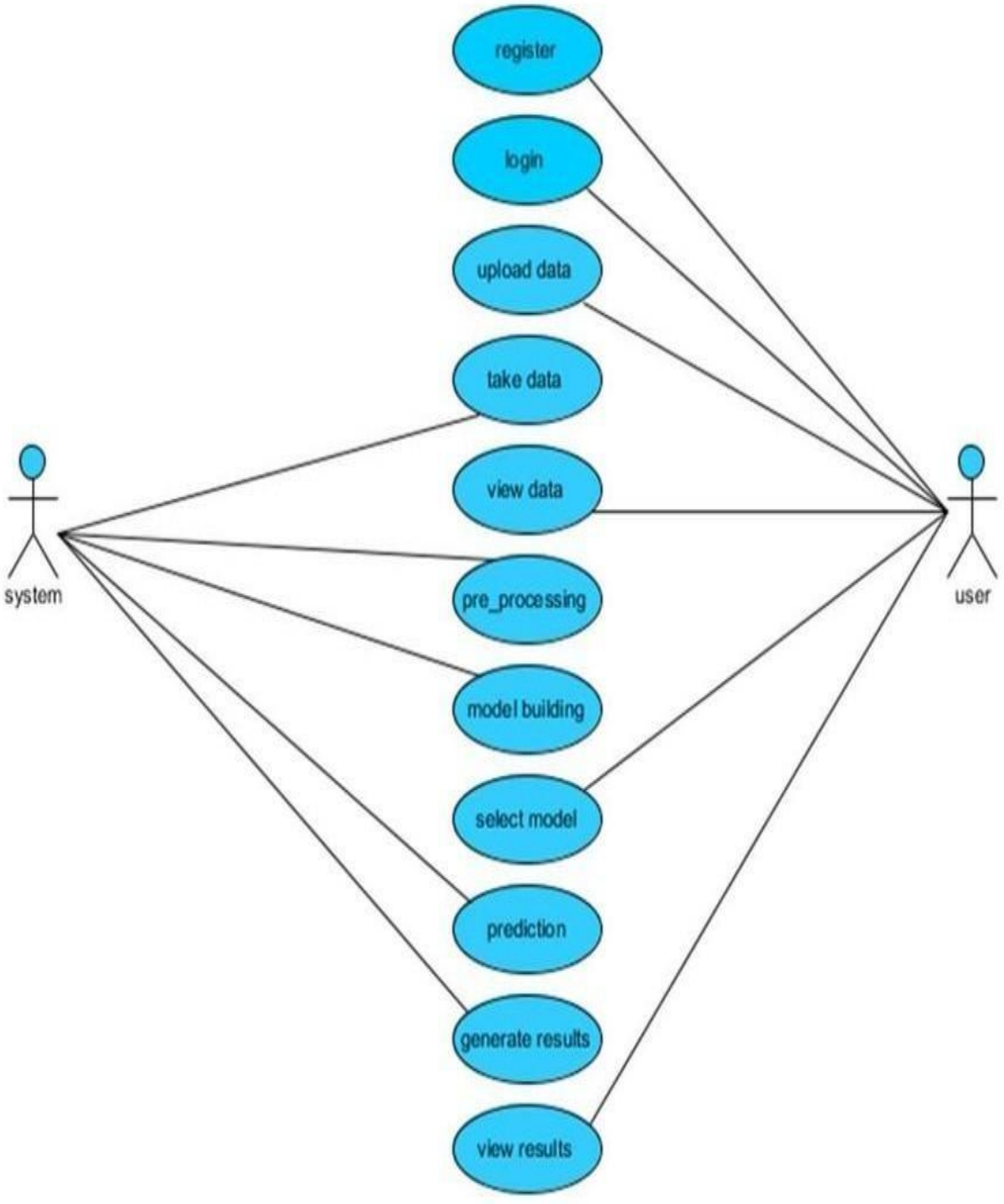


Figure 5.1: Use Case Diagram

5.1.2 Class Diagram

A Class Diagram is a structural UML diagram that represents the blueprint of a system by defining its classes, attributes, methods (functions), and relationships between different components. It provides a high-level view of the system's structure, helping developers understand how data and behaviors are organized within the system.

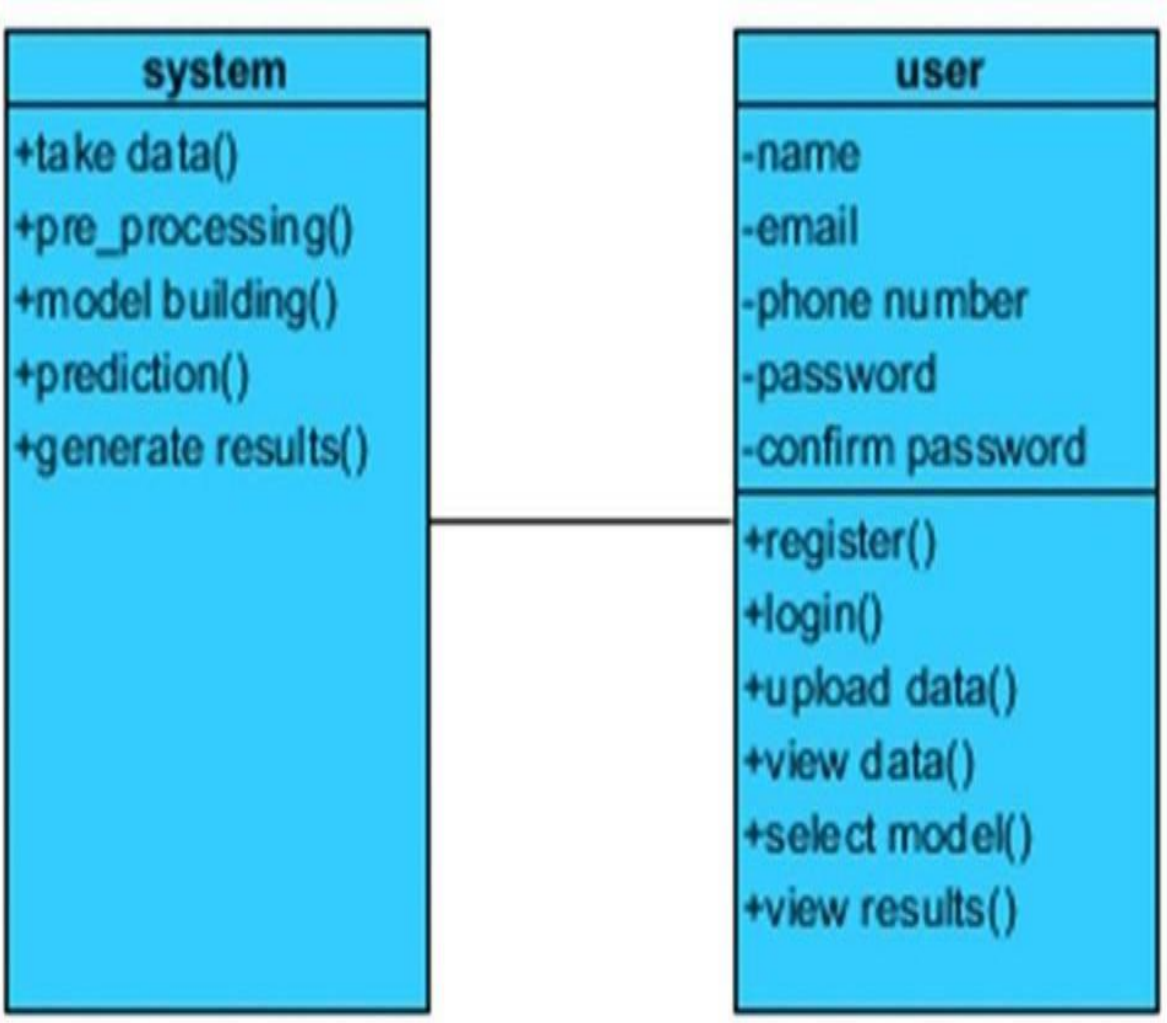


Figure 5.2: Class Diagram

5.1.3 Sequence Diagram

A Sequence Diagram is a type of UML (Unified Modeling Language) diagram that illustrates how different system components (objects, actors, or entities) interact with each other over time. It represents the sequence of messages exchanged between system components in chronological order, making it essential for understanding the flow of execution in a system.

Sequence diagrams are derived from Message Sequence Charts (MSC) and are sometimes referred to as event diagrams or timing diagrams because they showcase the timing and order in which messages are passed. They are widely

used in system design, software development, and modeling interactions in real-time systems.

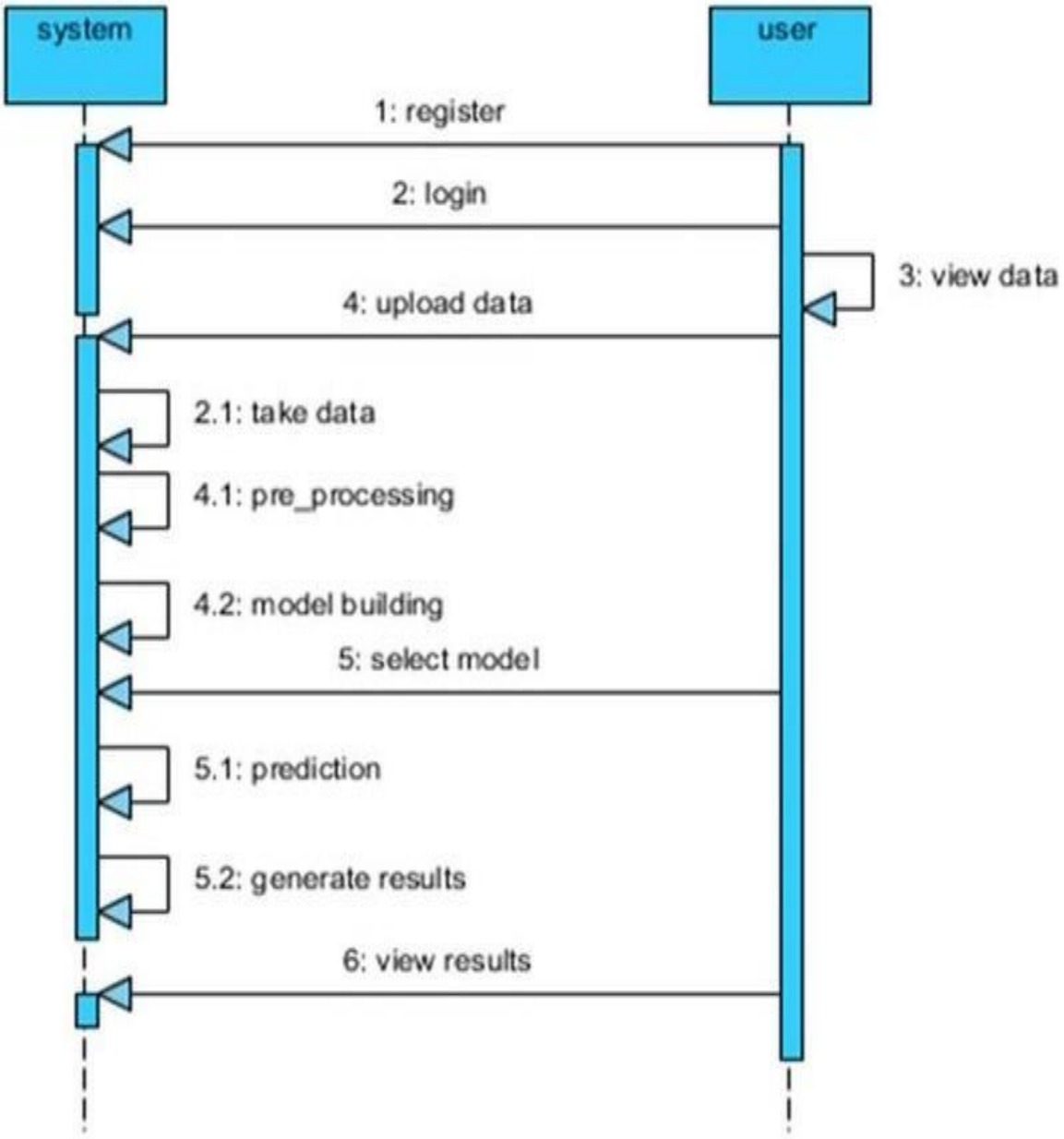


Figure 5.3: Sequence Diagram

5.1.4 Collaboration Diagram

A Collaboration Diagram (also known as a Communication Diagram) is a type of UML interaction diagram that illustrates how objects interact to achieve a specific task. It is similar to a Sequence Diagram, but instead of

focusing solely on the sequence of method calls, it emphasizes the structural organization of objects in the system and how they collaborate through message passing.

In a Collaboration Diagram, interactions are represented using numbered messages, indicating the sequence in which method calls occur. This numbering technique helps track the order of operations while also visually representing the relationship and connectivity between objects. Unlike Sequence Diagrams, which focus on time-based interactions, Collaboration Diagrams highlight the spatial arrangement of objects and their communication flow.

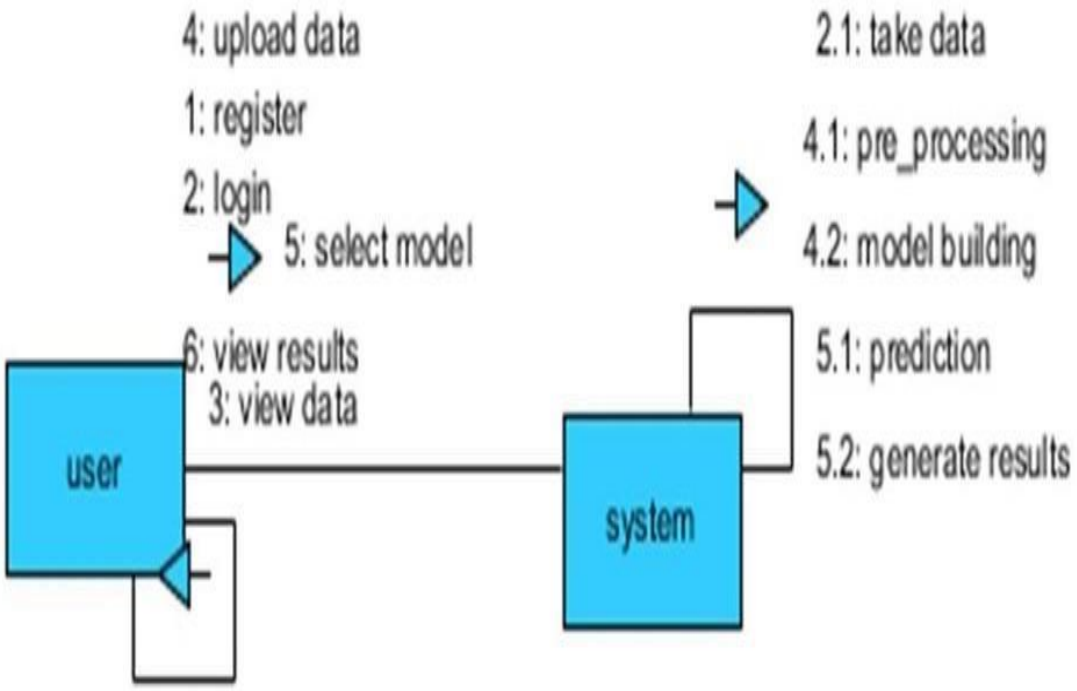


Figure 5.4: Collaboration Diagram

5.1.5 Activity Diagram

An Activity Diagram is a type of UML behavioral diagram that graphically represents the workflow of a system. It shows the step-by-step flow of activities, including decision points, loops (iterations), parallel processes (concurrent actions), and overall control flow. Unlike Sequence Diagrams, which emphasize interactions between objects over time, Activity Diagrams focus on process flows and execution logic, making them

ideal for modeling business processes, algorithms, and system functionalities.

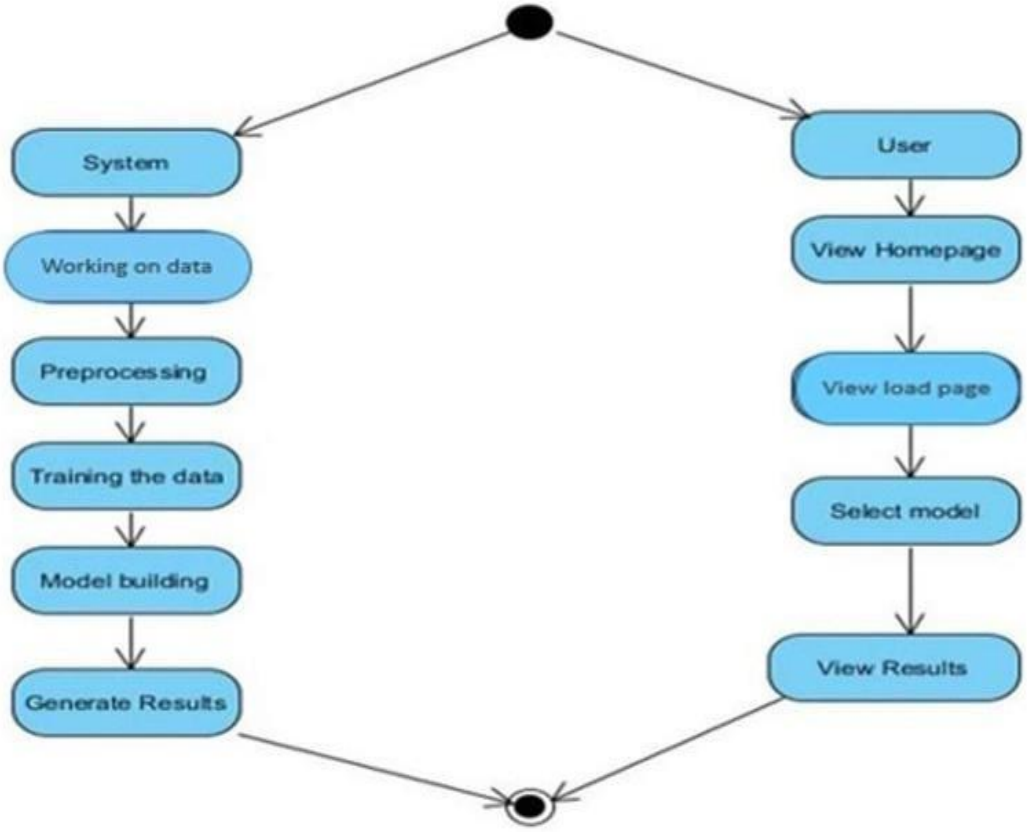


Figure 5.5: Activity Diagram

5.2 System Architecture

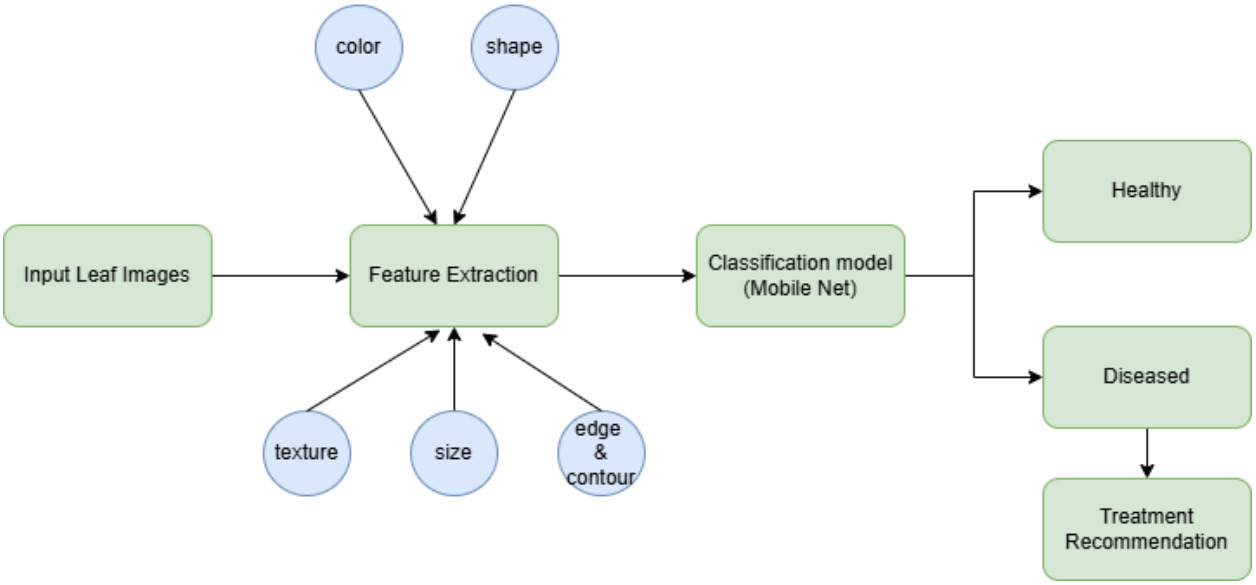


Figure 5.6: System Architecture

5.3 Flowchart

Flowcharts in machine learning algorithms visually represent the sequential steps involved in training and deploying models. For our medicinal plant identification project, the flowchart provides a structured visualization of the entire pipeline, starting from data acquisition to real-time prediction. The process begins with data collection, where a dataset of high-resolution images representing 40 medicinal plant species is gathered and loaded using TensorFlow's image dataset utilities. In the data preprocessing stage, images are resized to 224×224 pixels, normalized, and augmented with random flips and rotations to enhance dataset variability and improve model generalization. The dataset is then split into training, validation, and testing subsets in an 80:10:10 ratio for robust model evaluation.

The next phase focuses on model training, where multiple deep learning architectures—a custom Convolutional Neural Network (CNN), MobileNet model—are implemented. Hyperparameters such as learning rate, batch size, and the Adam optimizer are applied, with Sparse Categorical Crossentropy as the loss function. Additionally, early stopping is integrated to prevent overfitting and improve training efficiency. Once training is complete, the model evaluation stage computes key performance metrics, including accuracy, precision, recall, and F1 scores, using the test dataset. Finally, in the deployment and inference stage, the best- performing model is integrated into a web application using Django, allowing users to upload plant images and receive real-time identification results, along with details such as the confidence score and treatment recommendations.

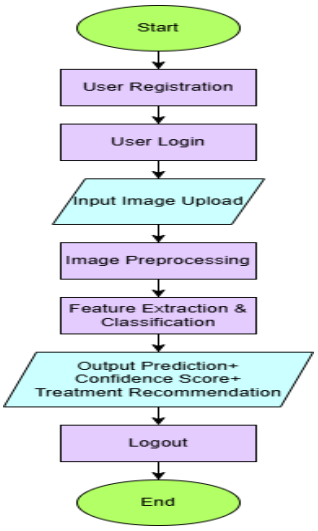


Figure 5.7: Flowchart of the system

CHAPTER 6

IMPLEMENTATION

In this project, there are two primary objectives:

Objective 1:

Enhance the dataset quality and build robust deep learning models for detecting diseases in potato leaves.

Objective 2:

Develop and deploy a user-friendly web application that enables real-time disease prediction by processing uploaded leaf images and providing detailed diagnostic feedback. The system comprises two major modules. The first module focuses on data preparation and model training. The second module is the web-based interface that allows users to interact with the system and receive predictions.

6.1 Working Flow of the System

1. **User Registration and Login:**
   Users begin by registering with the system and then logging in securely using their credentials. This ensures that only authorized users access the platform.

2. **Image Upload:**
   Once logged in, users are directed to a dashboard where they can upload images of plant leaves. The interface is designed to be simple and intuitive.

3. **Image Preprocessing:**
   Upon upload, images undergo preprocessing steps which include resizing to 128×128 pixels, normalization to a [0,1] scale, and augmentation using random flips and rotations. These steps ensure that the images are standardized and the model is exposed to various real-world conditions.

4. **Model Inference:**
   The preprocessed image is then fed into the trained deep learning model. Depending on the experiment, the system utilizes one of several models (custom CNN, MobileNet) to determine whether the leaf is healthy or

diseased.

5. **Result Display:**

   The system outputs a prediction along with additional diagnostic details, such as the scientific name of the plant, disease information, and treatment recommendations. Visual indicators of model confidence may also be displayed to help users understand the reliability of the prediction.

6.2 Datasets

The dataset is critical for the success of our system. We use a publicly available collection comprising 2,152 images of potato leaves from 3 different species. This dataset is carefully curated, including both healthy and diseased leaves, to ensure comprehensive training. Data augmentation—such as random flips, rotations, and scaling—is applied to simulate natural variability in the field, thereby enhancing the model's ability to generalize. The dataset is divided into training, validation, and testing subsets, typically in an 80:10:10 ratio, to ensure robust performance evaluation.

6.3 Data Pre-Processing

Data pre-processing transforms raw images into a format suitable for deep learning models and includes several essential stages:

6.3.1 Data Cleaning:

Data cleaning involves the removal of low-quality, corrupted, or improperly labeled images from the dataset. Techniques such as noise reduction and artifact removal are applied to enhance image clarity. Ensuring the dataset is clean is critical for model performance, as it reduces the chances of learning from erroneous data.

6.3.2 Data Integration:

Data integration consolidates images collected from multiple sources into a single, cohesive dataset. This process includes ensuring that all images are uniformly labeled and formatted, which is crucial for consistency during model training. By merging data from diverse sources, the system is exposed to a wide variety of disease manifestations and environmental conditions, enhancing its robustness.

6.3.3 Data Reduction:

Given the large size of the dataset, data reduction techniques are applied to streamline the information without compromising critical features. Methods like feature selection and dimensionality reduction (e.g., principal component analysis) are employed to retain essential characteristics such as texture, color patterns, and shape descriptors. This reduces computational complexity, speeds up model training, and prevents overfitting by eliminating redundant or irrelevant data.

6.4 System Implementation

The integration of these processes forms the backbone of the Potato Leaf Disease Detection system. The workflow begins with thorough data pre-processing to ensure high- quality input, followed by extensive model training using various deep learning algorithms. Once a model with optimal performance is selected—based on metrics such as accuracy, precision and recall—it is integrated into a Django- based web application. This application provides a seamless user experience, allowing users to quickly upload images and obtain accurate diagnostic predictions. By combining meticulous data preparation with state-of-the-art deep learning techniques, our system provides a robust framework for real-time disease detection in Potato leaves. The comprehensive approach—from data cleaning to user feedback—ensures that the system not only achieves high predictive accuracy but also remains scalable and user- friendly for practical field applications.

CHAPTER 7

TESTING

Software testing is an essential process to evaluate the quality and performance of the Potato Leaf Disease Detection system. It provides an objective view to stakeholders, ensuring that the application meets design and performance standards and is fit for its intended use. By executing the software and identifying bugs or defects, testing verifies that the product delivers accurate predictions and a seamless user experience. Testing in our project assesses aspects such as adherence to design requirements, correct responses to diverse inputs, efficient performance within acceptable timeframes, and overall system usability. Below are the key testing phases implemented for our system.

7.1 Functionality Testing

Functionality testing ensures that all individual components of the system operate as expected. For our project:

- The system correctly loads and preprocesses the 2,152-image dataset.
- The deep learning models (custom CNN, MobileNet) are successfully invoked and return predictions.
- The web application properly accepts image uploads and displays the prediction results, including diagnostic details such as the scientific name and potential treatments.
- User registration and login functionalities work seamlessly.
- Appropriate error messages are displayed when invalid inputs are provided.
- The flow from image upload to result display is accurate, timely, and consistent with the system's specifications.

7.2 Usability Testing

Usability testing evaluates the system's user interface and overall user experience:

- The application's interface is intuitive, enabling users to easily navigate between pages.
- The image upload process is straightforward and user-friendly.

- Feedback messages and instructions are clear, ensuring that users understand how to interact with the system.
- The prediction results and additional diagnostic information are presented in a comprehensible and visually appealing manner.
- Users can quickly locate help and support features within the application, if needed.

7.3 Interface Testing

Interface testing verifies the interaction between the system's frontend and backend components:

- The Django web application successfully connects with the deep learning model hosted on the local server.
- In case of connectivity issues or server interruptions, the application displays appropriate error messages.
- The communication between the client's web browser and the server is secure and reliable.
- Data is consistently transmitted between the image upload interface the preprocessing module, and the prediction engine.

7.4 Performance Testing

Performance testing measures the system's responsiveness and stability under various conditions:

- The system processes image uploads and returns predictions within an acceptable time frame, even under moderate internet speeds.
- The application maintains a secure connection, and user data is stored and transmitted in a secured manner.
- The transition between screens (e.g., login, image upload, prediction result) is smooth and quick.
- Stress testing ensures that the system can handle multiple concurrent users without performance degradation.

CHAPTER 8

RESULTS

The web application is designed as a user-friendly platform for Potato Leaf Disease Detection . When new users first access the system, they are prompted to register by providing their email address and creating a secure password, ensuring data privacy and secure access. Once registered, users log in using their credentials to gain full access to the application's features. After logging in, users are directed to the main interface where they can upload images of plant leaves using an intuitive image upload tool. The system processes the uploaded images through pre- trained deep learning models, which analyze the leaf for signs of disease and provide detailed diagnostic feedback, including the plant's scientific name and suggested treatment options. This seamless integration of user registration, image upload, and automated analysis ensures that both researchers and practitioners can quickly obtain accurate, real- time predictions, facilitating informed decision-making in farming and agriculture.

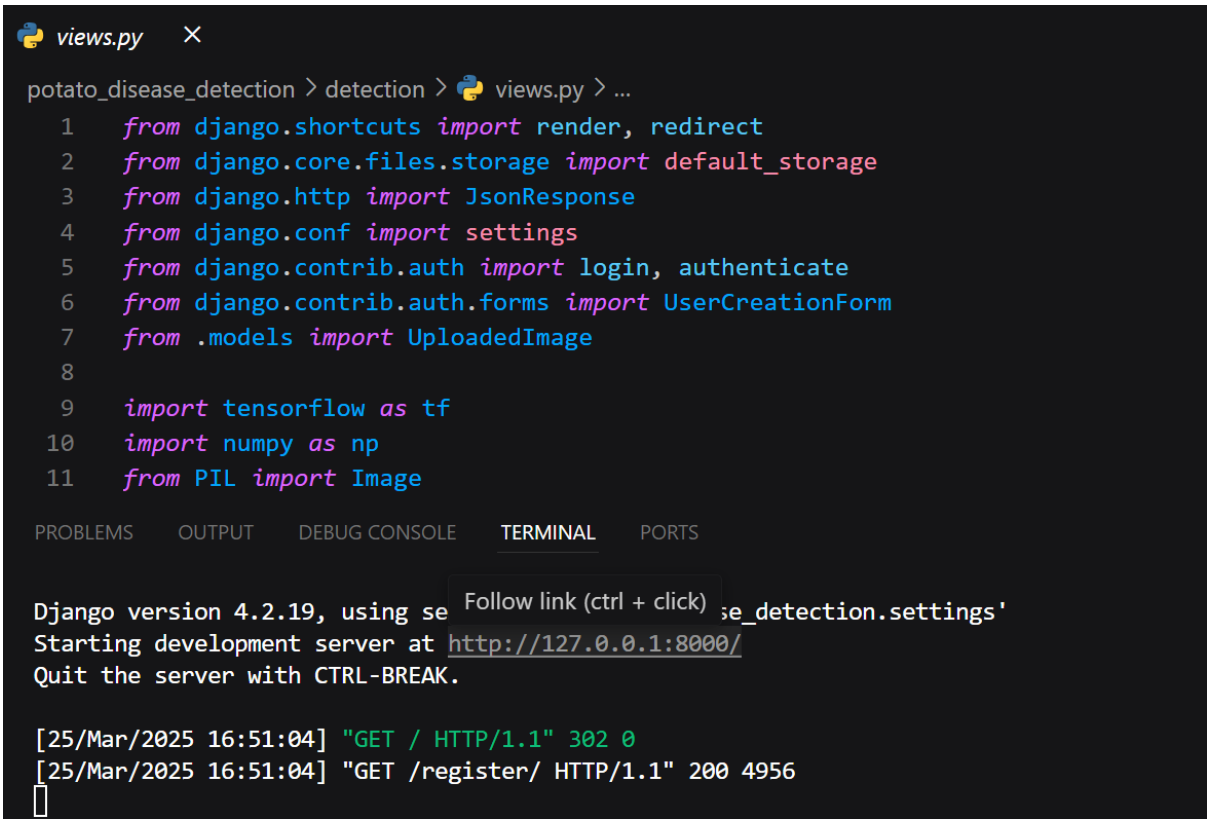


Figure 8.1: open local host link generated in vscode

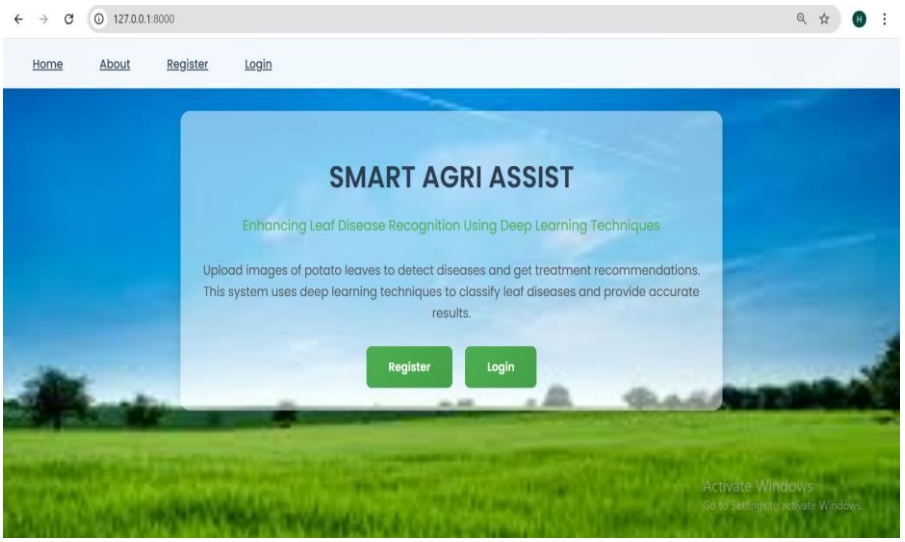


Figure 8.2 : Home Page

The above figure 8.2 displays the homepage of our project SMART AGRI ASSIST with options to register or log in. It introduces the system's purpose of detecting potato leaf diseases using deep learning techniques.

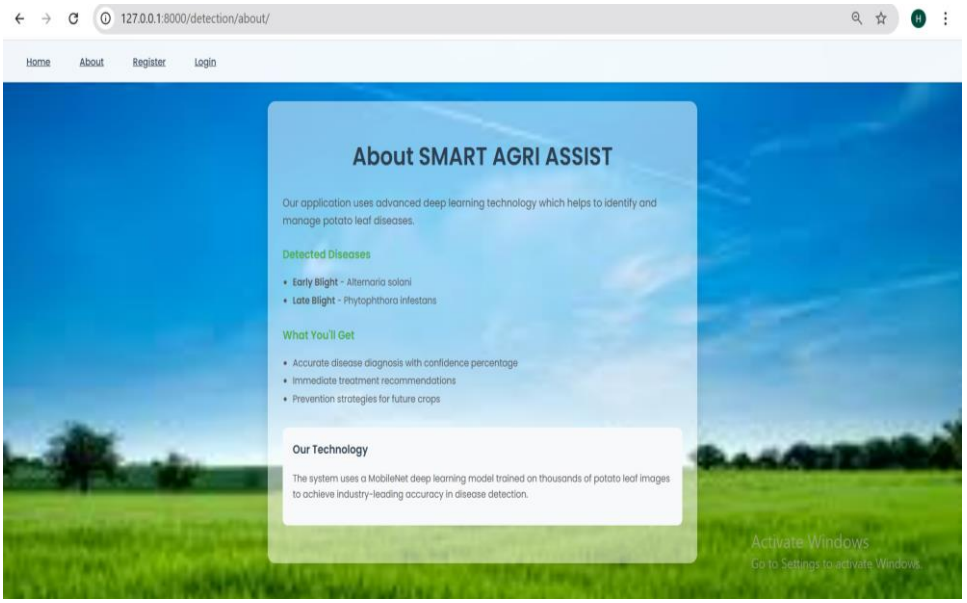


Figure 8.3 : About Page

The above figure 8.3 displays the about page of our project showing details of the system's features and detected diseases like Early and Late Blight. It highlights the use of MobileNet for accurate potato leaf disease classification and for giving treatment recommendations both in organic and chemical options.

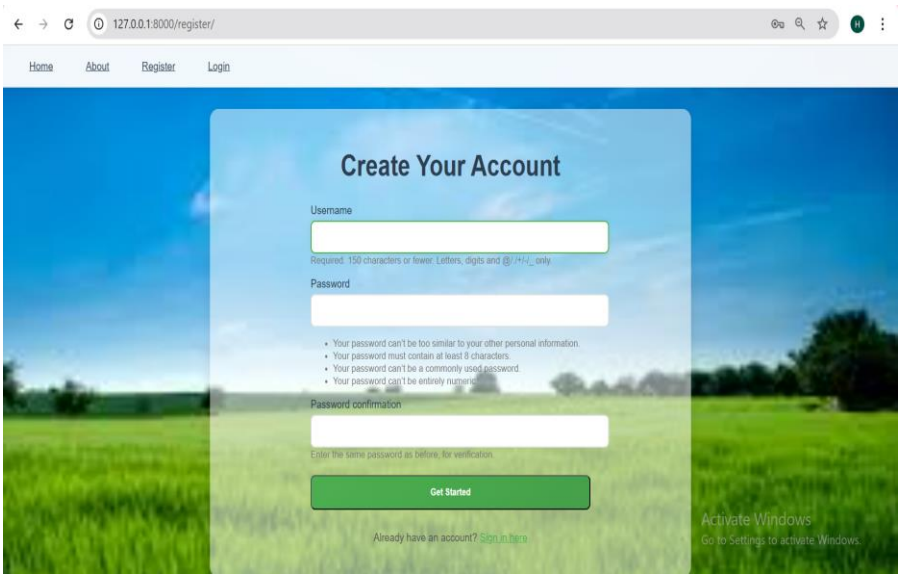


Figure 8.4 : Registration Page

The above figure 8.4 is displaying the registration page which allows new users to create an account by entering a username and secure password. It includes password validation rules and a "Get Started" button to complete registration.

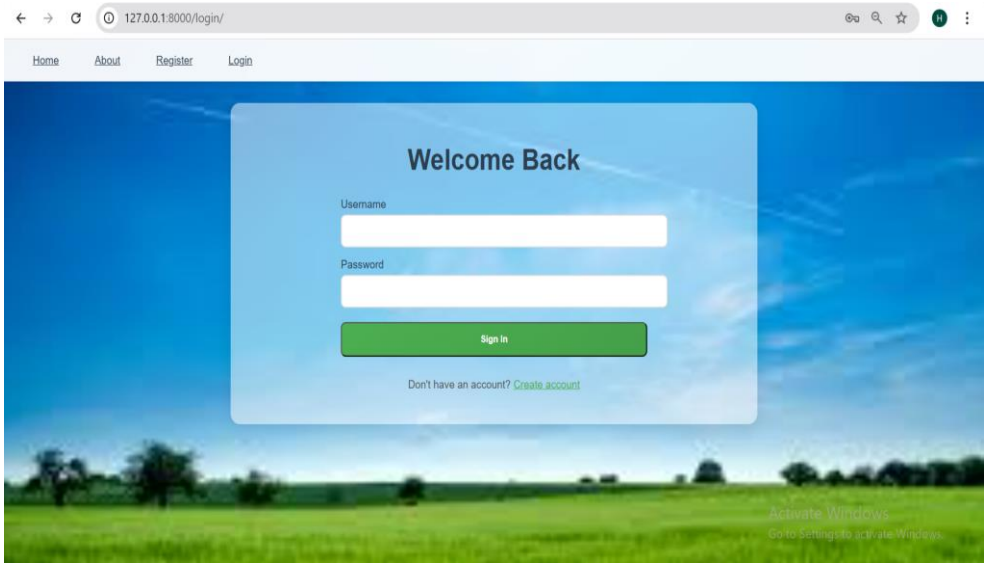


Figure 8.5 : Login Page

The above figure 8.5 is displaying the login screen which enables existing users to sign in using their credentials.It features a simple interface with a "Sign In" button and a link to create an account.

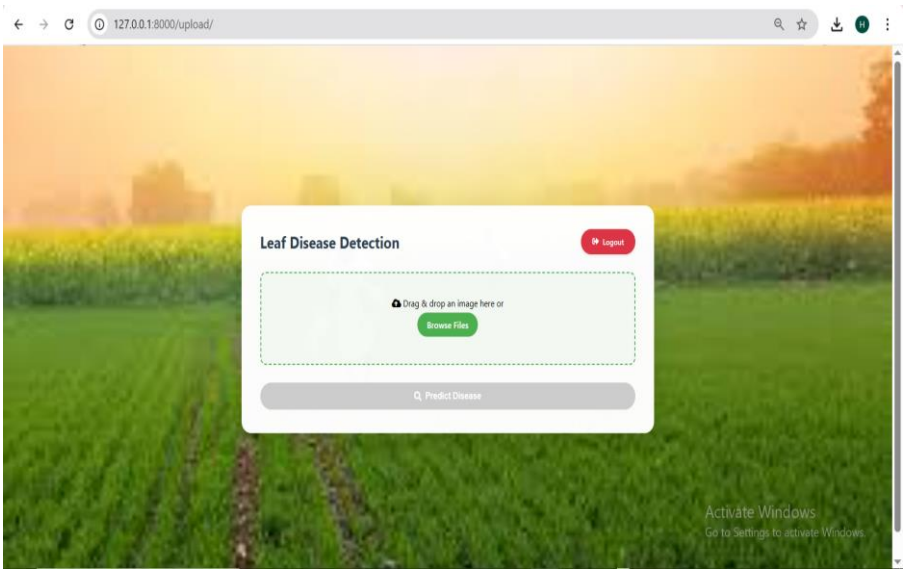


Figure 8.6 : Image Upload Page

The above figure 8.6 displays the image upload page which allows users to upload potato leaf images for disease detection. Users can drag & drop or browse files and click "Predict Disease" to analyze the image.

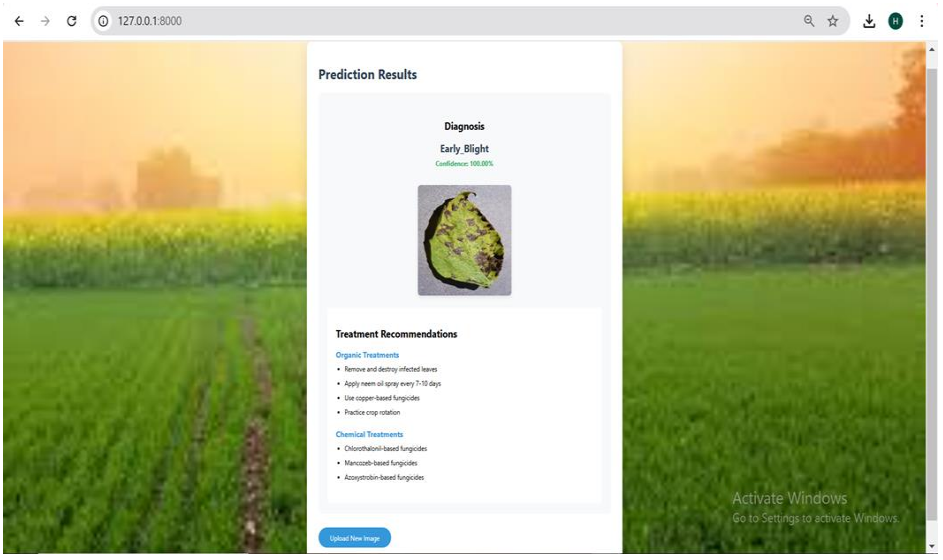


Figure 8.7 : Result Page

The above figure 8.7 displays the diagnosed disease (e.g., Early Blight) with a confidence score and image preview. It also provides organic and chemical treatment recommendations for the disease.

- **MobileNet**

The graph below illustrates the learning process of the deep learning system designed for classifying potato diseases over multiple epochs. In early stages, the training curve rises sharply, indicating effective feature extraction and optimization of gradients. Similarly, the model's accuracy on the validation set climbs over time, suggesting that model performs well on new, unseen data. Around the 10th epoch, the accuracy stabilizes, indicating that the model has learned the essential features for classification and is minimizing errors. The validation curve shows only minor fluctuations, indicating that the dataset is not overly complex and that overfitting is minimal, as the validation accuracy remains close to the training accuracy. The narrow gap between the two curves suggests that the model is well- regularized and that the hyperparameters have been effectively tuned.

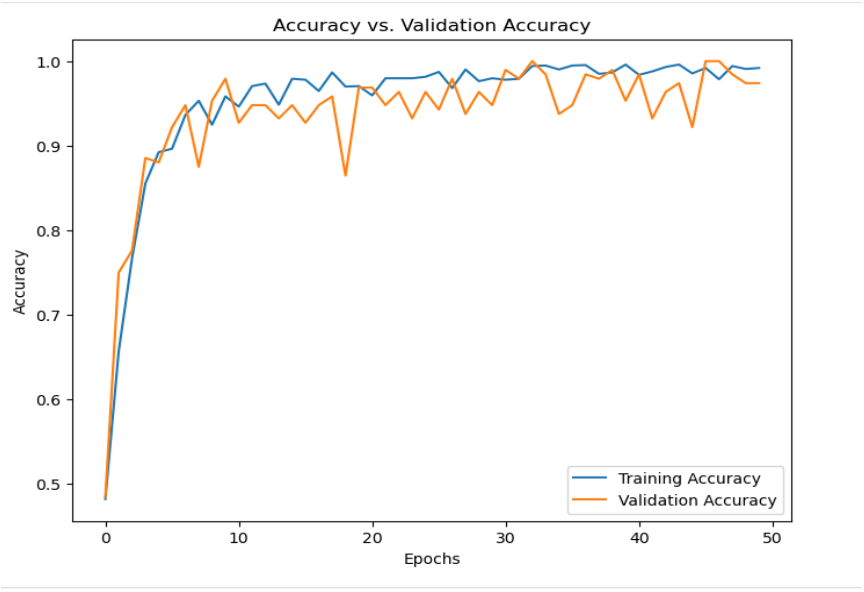


Figure 8.8 : Accuracy Trends During Training and Validation

The graph above depicts the training progression of the deep learning system designed for potato disease classification. At the outset, the rapid drop in both losses demonstrating the model's ability to learn and adapt to the data through successful parameter updates via backpropagation and gradient descent. After around 10 epochs, the training loss stabilizes at a low value, signaling convergence. The validation loss exhibits minor fluctuations, likely due to the nature of the validation dataset. The tight convergence of losses confirms the

model's robustness and reliability. These loss patterns highlight the model's effectiveness in achieving a well-regularized learning process for identifying agricultural diseases.

Figure 8.9 : Training and Validation Loss Over Epochs

This matrix evaluates performance of deep learning framework in classifying potato diseases by comparing predicted labels against actual labels. It assesses classification accuracy and identifies instances of misclassification across the categories. The model accurately identifies 110 healthy samples, 125 late blight samples, and 17 early blight samples, showcasing a high true positive rate. There are only a few misclassifications showing minimal confusion. The absence of false positives in the Healthy category highlights the model's effectiveness in recognizing non-infected leaves. Overall, the model exhibits impressive performance making it a dependable solution for automated detection of potato diseases in agriculture.

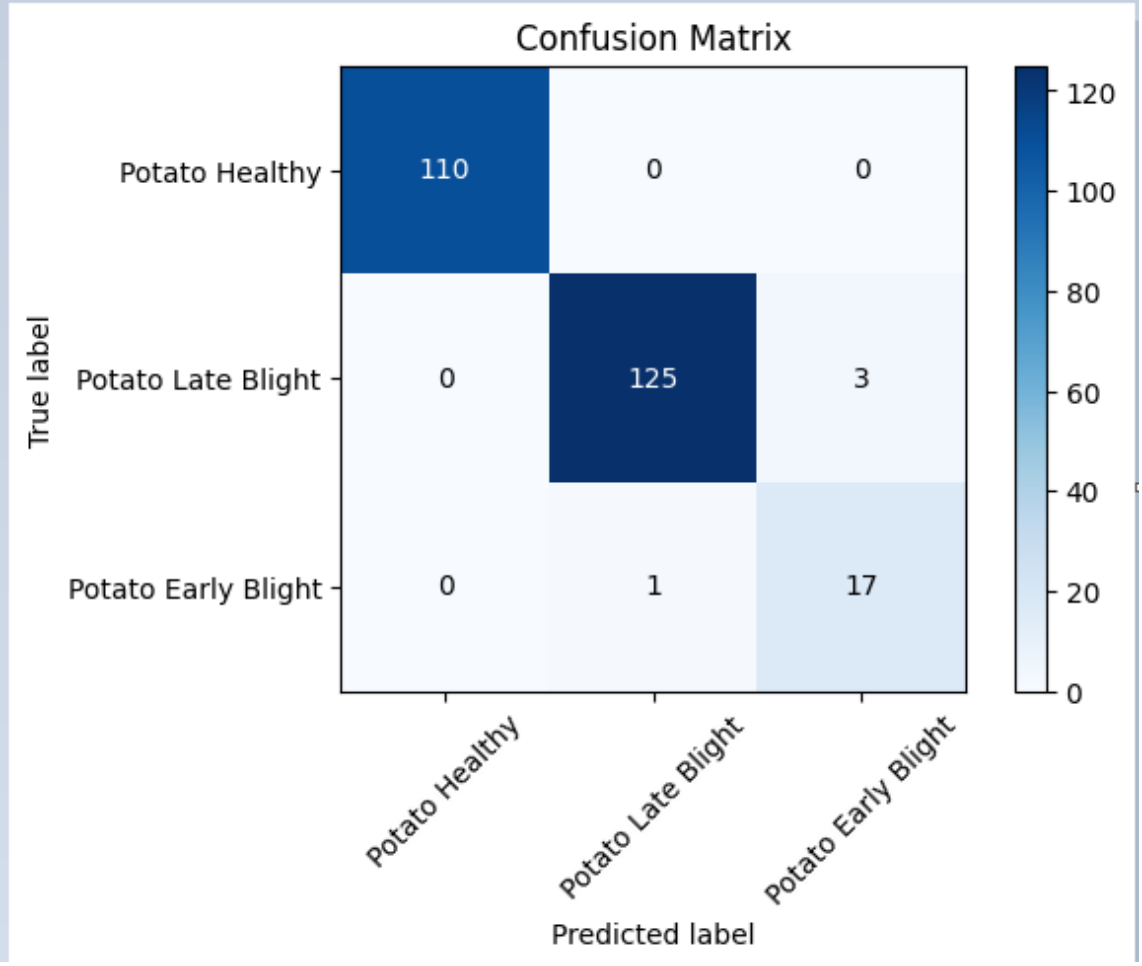


Figure 8.10 : Confusion Matrix for MobileNet

CONCLUSION

This research presents an advanced deep learning solution for potato disease detection using an optimized MobileNet-based CNN architecture. The model achieves exceptional 99% training accuracy by leveraging depthwise separable convolutions that maintain high performance while minimizing computational requirements, making it ideal for real-time agricultural applications. Our comprehensive dataset includes diverse examples of healthy leaves along with Early Blight and Late Blight infections, with data augmentation techniques enhancing the model's ability to recognize diseases under varying conditions. The system provides farmers with immediate diagnostic results through an intuitive Django-based web interface, complete with confidence metrics and tailored treatment suggestions for both organic and conventional farming practices.

The project demonstrates significant potential to transform traditional disease management approaches through accessible AI technology. Future developments will focus on expanding the model's capabilities to identify additional potato diseases and environmental stress factors, while maintaining its current efficiency for field deployment. We are also exploring integration with mobile platforms for offline functionality and the incorporation of real-time monitoring features through IoT devices in farm settings. These advancements will further bridge the gap between cutting-edge technology and practical farming needs, ultimately contributing to improved crop yields and sustainable agricultural practices worldwide.

.

REFERENCES

[1] Liu, J., Cheng, Q., Gong, W., et al. (2022). Deep learning-based tomato and potato diseases recognition. Journal of Food Engineering, 357, 109771.

[2] Arshaghi, A., Ashourian, M. & Ghabeli, L. (2023). Potato diseases detection and classification using deep learning methods. Multimed Tools Appl, 82, 5725–5742.

[3] Kumar, A., Patel, V.K. (2023). Classification and identification of disease in potato leaf using hierarchical based deep learning convolutional neural network. Multimed Tools Appl, 82, 31101–31127.

[4] Sharma, A., Zhang, L., & Tanwar, S. (2021). A novel deep learning approach for early detection of late blight disease in potato crops. Computers and Electronics in Agriculture, 182, 106075.

[5] Yuan, D., Wu, C., & Li, J. (2020). Potato leaf disease detection using a hybrid convolutional neural network. IEEE Access, 8, 72671-72677.

[6] Kong, G., Wang, H., Wang, L., et al. (2022). Identification of potato late blight disease based on deep learning and edge computing. IEEE Internet of Things Journal, 9(6), 5046-5054.

[7] Shrestha, R., Gaire, A., & Moh, S. (2021). An efficient potato disease classification model using convolutional neural networks. In Proceedings of the IEEE International Conference on Computer Vision (ICCV) Workshops, 2021.

[8] Parihar, N., Rani, A., Gupta, M., et al. (2020). Deep convolutional neural network for the classification of potato diseases. In Proceedings of the International Conference on Computer Vision and Pattern Recognition (CVPR), 2020.

[9] Zhang, L., Zhang, Y., & Zhu, Z. (2022). Potato disease detection using deep neural networks. Computers in Industry, 134, 103590.

[10] Yang, J., Xie, Y., & Wei, J. (2021). Potato leaf disease detection using a novel CNN architecture. Computers and Electronics in Agriculture, 184, 106095.

[11] Du, J., Ma, X., & Li, B. (2022). Potato leaf disease classification using a deep convolutional neural network with attention mechanism. IEEE Transactions on Image Processing, 31, 173-183.

[12] Zhang, Y., Chen, S., & Sun, Q. (2020). Detection of potato late blight disease using a convolutional neural network-based method. Plant Disease, 104(10), 2671-2678.

[13] Sharma, S., Sharma, A., & Gupta, A. (2021). A survey of recent advances in potato disease detection using deep learning techniques. Journal of Agricultural Science and Technology, 23(4), 815-834.

[14] Wang, L., Liu, L., & Li, Y. (2020). An efficient deep learning model for early detection of potato diseases using CNN. Journal of Agricultural Information Science, 42(2), 59-69.

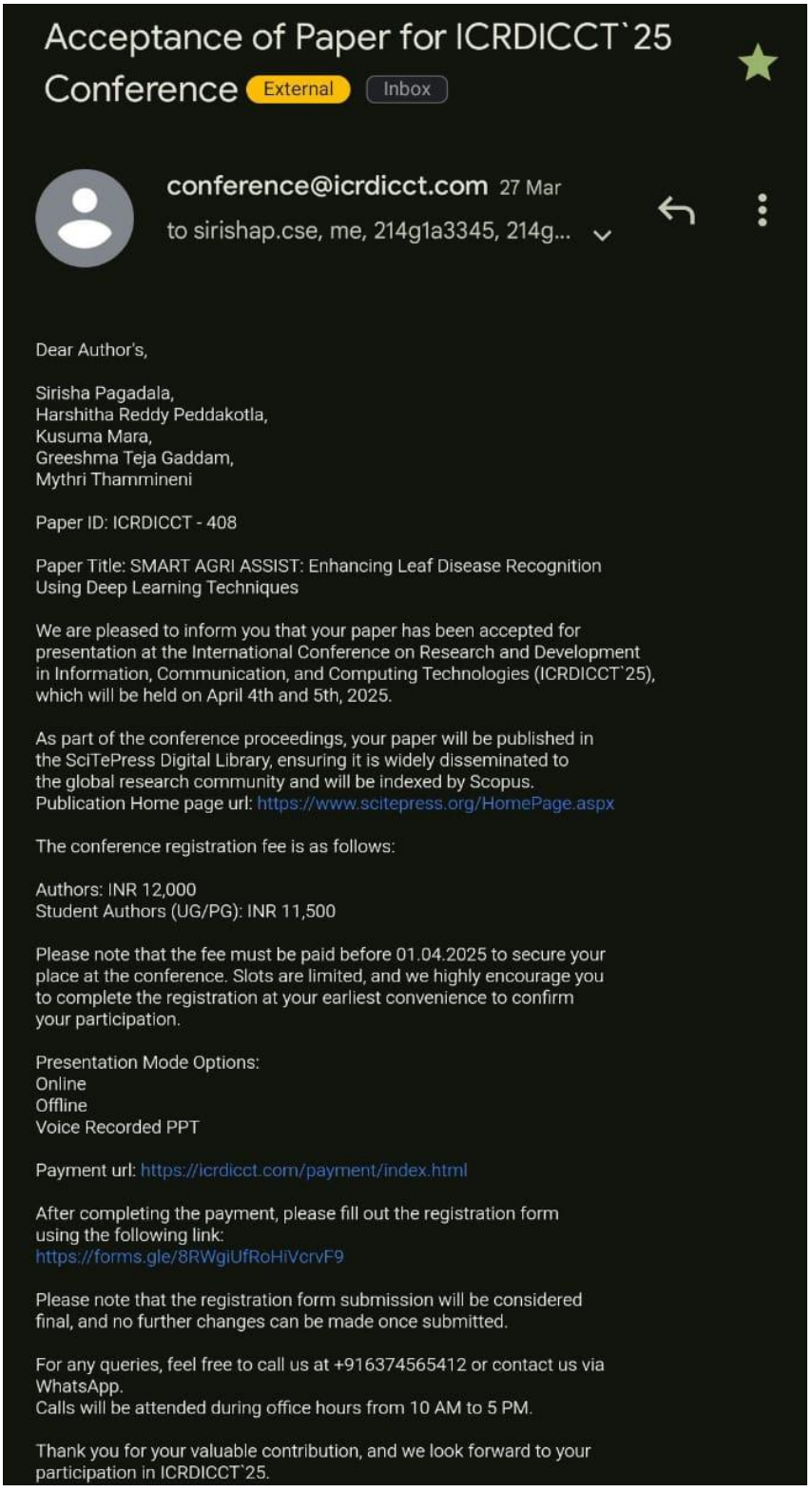
[15] Ma, R., Hu, J., & Li, Y. (2022). A novel method for the identification of potato late blight disease based on convolutional neural networks. Computers in Industry, 152, 103488.

[16] Shen, L., Zhang, J., & Huang, X. (2021). Potato disease recognition using a deep learning-based approach. Plant Pathology, 70(4), 943-950.

[17] Li, M., Yang, Z., & Wang, Y. (2020). Automated detection of potato diseases using convolutional neural networks. International Journal of Agricultural and Biological Engineering, 13(6), 103-11.

Paper Publication

Acceptance Mail



Acceptance of Paper for ICRDICCT`25 Conference External Inbox

conference@icrdicct.com 27 Mar
to sirishap.cse, me, 214g1a3345, 214g...

Dear Author's,

Sirisha Pagadala,
Harshitha Reddy Peddakotla,
Kusuma Mara,
Greeshma Teja Gaddam,
Mythri Thammineni

Paper ID: ICRDICCT - 408

Paper Title: SMART AGRI ASSIST: Enhancing Leaf Disease Recognition Using Deep Learning Techniques

We are pleased to inform you that your paper has been accepted for presentation at the International Conference on Research and Development in Information, Communication, and Computing Technologies (ICRDICCT`25), which will be held on April 4th and 5th, 2025.

As part of the conference proceedings, your paper will be published in the SciTePress Digital Library, ensuring it is widely disseminated to the global research community and will be indexed by Scopus.
Publication Home page url: https://www.scitepress.org/HomePage.aspx

The conference registration fee is as follows:

Authors: INR 12,000
Student Authors (UG/PG): INR 11,500

Please note that the fee must be paid before 01.04.2025 to secure your place at the conference. Slots are limited, and we highly encourage you to complete the registration at your earliest convenience to confirm your participation.

Presentation Mode Options:
Online
Offline
Voice Recorded PPT

Payment url: https://icrdicct.com/payment/index.html

After completing the payment, please fill out the registration form using the following link:
https://forms.gle/8RWgiUfRoHiVcrvF9

Please note that the registration form submission will be considered final, and no further changes can be made once submitted.

For any queries, feel free to call us at +916374565412 or contact us via WhatsApp.
Calls will be attended during office hours from 10 AM to 5 PM.

Thank you for your valuable contribution, and we look forward to your participation in ICRDICCT`25.

Certificates

SCITEPRESS DIGITAL LIBRARY Scopus

ICRDICCT CONFERENCE PROCEEDINGS 2025.

International Conference on

Research and Development in Information, Communication and Computing Technologies (ICRDICCT'25)

This is to certify that

Kusuma Mara

has successfully presented the paper titled

SMART AGRI ASSIST: Enhancing Leaf Disease Recognition Using Deep Learning Techniques

in Proceedings of the International Conference on Research and Development in Information, Communication and Computing Technologies (ICRDICCT'25), held on April 4 & 5, 2025, at E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India.

Organizing Secretary	Industry Partner	Principal
Dr.S.Manikandan	Ms.Gugapriyaa Sivakumar NTL Technology	Dr.S.Ramabalan

ORGANIZED BY

E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India.

&

NTL Technology Erode, Tamil Nadu, India.

NTLTECHNOLOGY
New Thinking and Learning

SCITEPRESS DIGITAL LIBRARY Scopus

ICRDICCT CONFERENCE PROCEEDINGS 2025.

International Conference on

Research and Development in Information, Communication and Computing Technologies (ICRDICCT'25)

This is to certify that

Greeshma Teja Gaddam

has successfully presented the paper titled

SMART AGRI ASSIST: Enhancing Leaf Disease Recognition Using Deep Learning Techniques

in Proceedings of the International Conference on Research and Development in Information, Communication and Computing Technologies (ICRDICCT'25), held on April 4 & 5, 2025, at E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India.

Organizing Secretary	Industry Partner	Principal
Dr.S.Manikandan	Ms.Gugapriyaa Sivakumar NTL Technology	Dr.S.Ramabalan

ORGANIZED BY

E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India.

&

NTL Technology Erode, Tamil Nadu, India.

NTLTECHNOLOGY
New Thinking and Learning


SCITEPRESS DIGITAL LIBRARY
Scopus
ICRDICCT CONFERENCE PROCEEDINGS 2025.
International Conference on
Research and Development in Information, Communication and Computing Technologies
(ICRDICCT'25)
This is to certify that
Mythri Thammineni
has successfully presented the paper titled
SMART AGRI ASSIST: Enhancing Leaf Disease Recognition Using Deep Learning Techniques
in Proceedings of the International Conference on Research and Development in Information, Communication and Computing Technologies (ICRDICCT'25), held on April 4 & 5, 2025, at E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India.
Organizing Secretary
Dr.S.Manikandan
Industry Partner
Ms.Gugapriyaa Sivakumar
NTL Technology
Principal
Dr.S.Ramabalan
ORGANIZED BY
E.G.S. Pillay Engineering College, Nagapattinam, Tamil Nadu, India.
&
NTL Technology
Erode, Tamil Nadu, India.
NTLTECHNOLOGY
New Thinking and Learning